

VLA Drone Master Thesis Guide

Litter search analysis and mapping

Foundations literature methodology and practical labs

A complete guide for a student beginning drone navigation and robotics

Prepared from Leandro's project brief

Single VLA edition | 26 September 2026

The project you will develop

A drone searches a field containing placed litter, inspects the items, describes visible evidence and produces a map. One compact vision language action model decides when to pause a normal survey, approach a visible candidate, inspect it, mark it and resume the unfinished route. The thesis tests the value of these decisions under a fixed mission and compute budget.

Your software sends bounded position or velocity commands. The existing autopilot remains responsible for stable flight. Begin with a classical baseline and imitation learning; use the Orin NX 16 GB for inference and a workstation for training.

One document from first concepts to the thesis

This guide contains the robotics foundations, selected literature review, simulator and dataset methodology, ten introductory labs, two research build labs, fourteen explanatory figures, deployment strategy, transfer evaluation, thesis writing guidance and reusable research records. Read the explanations before the linked exercise, then use its evidence when writing the relevant thesis chapter.

The code supplement contains executable files and example outputs. All core explanations and research guidance are in this volume. The laptop results are measured teaching examples; later simulator, GPU and hardware procedures state their validation status explicitly.

Student: _____ Supervisor: _____

Programme: _____ Submission date: _____

VUB-inspired A4 layout using the institutional blue and orange palette [38]. This guide does not certify faculty submission-template compliance. Use the approved faculty cover and official identity assets when submitting the thesis.

START HERE

How to use the complete guide

Work in two connected modes. In learning mode, use the explanations and labs to understand each component. In research mode, freeze the interfaces, compare methods fairly and make claims supported by experiments. A successful tutorial is a starting point for the thesis, not its final scientific result.

Part of the work	What you receive	Status
Concepts and research design	Foundations, literature, methods, evaluation and writing guidance	Contained in this document
Laptop exercises	Toy simulator, data collection, tiny imitation learner and evaluation	Executed; 10 checks passed
Flight simulator integration	World and camera generator, recorder and square script	Source and syntax checked; lab execution required
SmolVLA movement pilot	GPU action training, data export and evaluation	Source-reviewed against v0.4.4; GPU smoke test required
Unified mission VLA	Decision, grounding and analysis heads; Labs 11 and 12	Specified research work; not implemented by the supplied trainer
Orin and real flight	Benchmark and transfer protocols with review gates	Proposed experiments; no hardware results claimed

Start with a manageable loop

Read the foundations and the six-paper reading route. Then follow the laptop labs before setting up PX4. The toy task is to centre above a requested object using an ideal camera and simple motion. It teaches data and action contracts. Realistic field search, perception and flight integration follow in later chapters.

Keep the code supplement beside this document

Unzip VLA_Drone_Starter_Kit and run commands from its root unless a lab states otherwise. Source files remain separate because you need to execute and modify them. The folder contains example logs and a validation record. The figure index links to fourteen visuals. Editable figure files accompany the guide in the student package.

Use supervision deliberately

You own the exercises and explanations. The supervisor helps define scope and research claims. A lab engineer supports the first simulator environment, time synchronisation and command-interface review. The pilot owns field operation. Bring a failing command, version record, log and expected behaviour when asking for help.

NAVIGATION

Contents and working route

Use the chapter route for concepts and the lab index for implementation. Labs 1–10 provide introductory exercises; Labs 11–12 specify the mission research implementation.

Ch.	Focus	Pages
1	Scope and foundations	5–12
2	Literature and research positioning	13–17
3	Action contracts and first learning loop	18–28
4	Flight simulation and the classical mission	29–42
5	Datasets and VLA training	43–56
6	Reasoning and onboard compute	57–61
7	Transfer and research evaluation	62–68
8	Project plan and thesis writing	69–75
9	Research records and glossary	76–78
	Bibliography	79–84

Practical lab index

- Lab 1 [Run a baseline episode](#) · [page 22](#)
- Lab 2 [Trace one observation and command](#) · [page 23](#)
- Lab 3 [Collect and validate demonstrations](#) · [page 24](#)
- Lab 4 [Train the tiny imitation policy](#) · [page 26](#)
- Lab 5 [Evaluate behaviour and language controls](#) · [page 27](#)
- Lab 6 [Start PX4 and Gazebo](#) · [page 29](#)
- Lab 7 [Add the field and camera](#) · [page 30](#)
- Lab 8 [Send position commands in SITL](#) · [page 32](#)
- Lab 9 [Adapt SmoVLA inputs and outputs](#) · [page 51](#)
- Lab 10 [Train and evaluate the VLA](#) · [page 52](#)
- Lab 11 [Build the mission manager](#) · [page 39](#)
- Lab 12 [Integrate the unified mission VLA](#) · [page 56](#)

Begin with the foundations and Lab 1. Build the mission manager after the baseline; add unified training after the movement pilot. Draft methods alongside saved experiments.

NAVIGATION

Figure index and visual learning route

Use these figures to explain the system aloud before implementing it. The captions identify measured teaching outputs and illustrative designs. Figure sources and editable SVG files are included in the student package.

- 1 The litter search mission · page 6
- 2 From camera pixels to ground offsets · page 10
- 3 One VLA decides and acts · page 11
- 14 Inside the single VLA · page 12
- 4 Anchor one prediction to one observation · page 20
- 5 A recorded teaching episode · page 23
- 6 Measured teaching results · page 28
- 7 The simulator and flight software loop · page 34
- 8 From repeated sightings to object tracks · page 42
- 9 Mission labels and information boundaries · page 48
- 10 Training one shared model · page 54
- 11 One model across mission modes · page 61
- 12 The stages of transfer evidence · page 65
- 13 Main system and decision comparisons · page 68

Use the visual with the adjacent explanation

Predict an action, sketch a message boundary or identify which experiment would support a claim. Then read the surrounding methods and run the corresponding exercise. A diagram is a guide to implementation, not a substitute for measured behaviour.

All fourteen figures were prepared for this guide; numbers identify files and remain stable between editions. Figures 5 and 6 use the supplied teaching outputs; the other figures explain the proposed system and worked examples. No image is presented as a real flight or a completed GPU experiment.

CHAPTER 1 SCOPE AND FOUNDATIONS

Define the single VLA thesis

The research target is one compact learned model that monitors a normal drone survey and decides when an inspection is worth a detour. It also predicts local movements and visually supported object labels. The model receives the mission instruction and current mission context throughout the flight.

Research questions

RQ1 Mission decisions. Does learned interruption, inspection completion and resumption increase correctly mapped litter under a fixed mission budget compared with a tuned rule-based mission?

RQ2 Actions and evidence. Can the same model ground a candidate in an image, generate bounded inspection movements and produce supported labels? Which contribution comes from decisions, motion or extra observations?

RQ3 Deployment and transfer. Can this unified model meet measured Orin NX 16 GB deadlines, and how do its errors change between simulation and independent real field sessions?

Scope for the first complete system

Include initially	Defer until this works
Flat open field, static litter and daylight.	Obstacles, steep terrain and aggressive flight.
Fixed altitude and yaw; horizontal position increments.	Learned altitude, yaw or velocity policy.
Classical survey route with a saved coverage record.	Learning the complete coverage planner.
One VLA for detection/grounding, decisions, motion and labels.	A second VLM or external learned detector in the target system.
Supervised demonstrations of complete mission events.	RL unless a specific later question requires it.

What remains ordinary software

The mission manager stores progress and applies valid transitions. Geometry projects observed image regions using calibration and estimated pose. The command supervisor checks proposals, and the existing autopilot maintains flight. These components do not independently choose which litter to inspect.

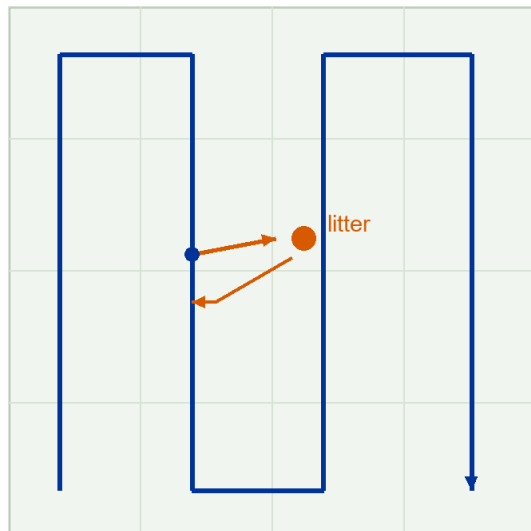
“Analyse” means category and visible attributes, with unknown allowed. “Far” means a small but observable candidate in the camera footprint; establish the actual range from camera geometry and pixel size. The first meeting must settle camera, positioning, training GPU, field access and pilot support.

CHAPTER 1 SCOPE AND FOUNDATIONS

Follow the survey with purposeful detours

The default behaviour is ordinary coverage. The VLA observes the field while that route executes, and proposes a detour only when the visible candidate and mission context justify one.

NORMAL SURVEY WITH ONE DETOUR



Blue: survey route

Orange: inspect and rejoin

Illustrative path; save coverage before detouring.

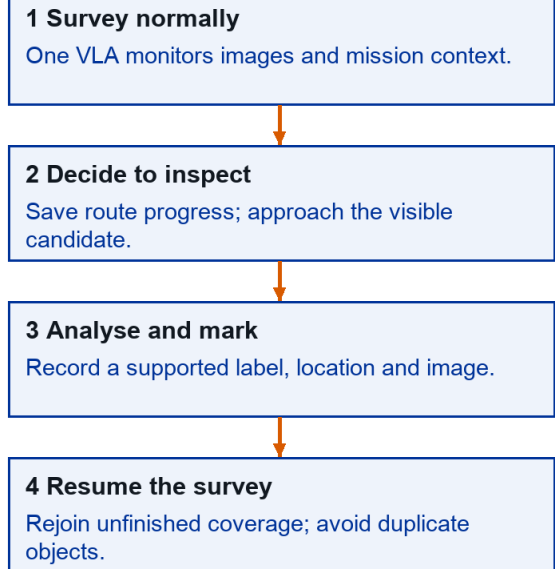


Figure 1. Illustrative survey, inspection and resumption. The route and object locations are schematic. Save route progress before taking the detour.

Follow one object through the mission

A small candidate becomes visible. DETOUR saves the route state. The VLA approaches with short bounded actions, obtains evidence, proposes MARK, then requests RESUME. The manager rejoins unfinished coverage and retains the mapped track to prevent duplicates.

A failed inspection still needs a defined outcome

If the target disappears, budget expires or evidence stays ambiguous, hold or resume with a recorded failure/unknown outcome. The mission should not remain trapped at one candidate. A supervisor override is logged and counted.

CHAPTER 1 SCOPE AND FOUNDATIONS

Robotics concepts you need first

Concept	Meaning and a drone example
State	What is known about the drone now: position, orientation, velocity and validity flags.
Observation	What the policy receives: a camera frame, timestamped estimated state and an instruction.
Action or setpoint	A request such as “go to north 2 m, east 1 m, down –5 m”. It is not a motor signal.
Policy	A rule or learned model that maps an observation to an action.
Planner	Chooses the next task: survey a cell, inspect an item, revisit or finish.
Autopilot	Estimates vehicle state and tracks the commanded motion using its existing control loops.
Closed loop	Observe, command, observe the result, then revise the next command.
Simulator	Computes vehicle motion, sensor readings and rendered images without flying a real drone.
SITL	The real autopilot software runs on a computer connected to a simulated vehicle.
Ground truth	A privileged reference used for labels or scoring; it is not an onboard measurement.

Walk through one inspection

The camera shows a bottle and a can. The instruction is “centre above the can”. The policy proposes a nearby horizontal target. The command checker accepts it, and PX4 moves towards it while holding altitude. The next image shows the can closer to the centre. After several updates, the system saves a close-up and returns control to the mission manager.

Separate the two computers

The **flight controller** runs the autopilot and remains responsible for stable flight. The **companion computer**, here the Jetson Orin, runs perception, mapping and learned models. A workstation runs the simulator and training. In deployment the simulator disappears, but the model and flight adapter retain the same contracts.

Exercise before using a model

Draw the path from camera to policy to setpoint to autopilot to motion. Mark which values are measured, predicted or ground truth. Then explain why a correct text answer about a bottle does not yet demonstrate a flight policy: the system must choose and execute useful actions from changing observations.

CHAPTER 1 SCOPE AND FOUNDATIONS

Coordinates units and time

Many apparent learning failures are frame or timing errors. Write these conventions in a single shared module and test them before recording demonstrations.

Frame	Axes and convention
Local NED	x = north, y = east, z = down, in metres relative to a recorded origin. Above the origin means negative z.
Local ENU	x = east, y = north, z = up. Common in robotics software and some simulation interfaces.
Body FRD	x = forward, y = right, z = down, attached to the drone. Forward changes direction as the drone yaws.
Camera optical	Typically x = image right, y = image down, z = viewing direction. Calibrate its rotation relative to the body.
Geographic	Latitude and longitude are angles, not local metres. GeoJSON coordinates are longitude then latitude.

NED from ENU: [north, east, down] = [y_ENU, x_ENU, -z_ENU]
Yaw: record radians internally; convert explicitly at the SDK boundary.

A position command example

If the local origin is on the ground, NED [3, 2, -5] means 3 m north, 2 m east and 5 m above that origin. It is not automatically 5 m above the terrain on a slope. The initial simulator uses a level ground plane so those heights agree.

A timestamp is part of an observation

Store camera capture time, frame arrival time, policy start and finish time, and command send time. Interpolate estimated pose at image capture time for mapping. Log the clock source and offset; a camera clock and an autopilot clock may differ. A receive timestamp alone includes transport delay. Training uses simulation time when simulating. Orin deployment tests also use monotonic wall time for command expiry. Accelerating or pausing the simulator must not conceal slow inference. Never match an image to a future measurement that would not be available at deployment.

Completion test

In simulation, command positive north, then positive east, then a small upward move. Overlay the position trace with the rendered view. Repeat a horizontal move after changing yaw by 90 degrees. Project four known ground points into the camera and back. Save the plots; all axis signs and rotations must agree.

CHAPTER 1 SCOPE AND FOUNDATIONS

Coordinates without guessing signs

A coordinate frame is a set of axes with an origin. A number such as “2” cannot be a flight command until you state the axis, unit and frame. The starter uses local north east down, abbreviated NED: positive north, positive east and positive down. At five metres above its ground origin, down equals –5 m.

Quantity	Toy convention	Common mistake
Image pixel	u grows right; v grows down	Treating image v as north
Horizontal action	[delta north, delta east] in metres	Sending pixels directly as metres
Height	Positive distance above the flat ground	Using negative height in projection
Gazebo world	East north up, ENU	Passing ENU directly to a NED interface
Yaw	Heading about the vertical axis	Assuming every real camera image is north up

Work one image to ground conversion

The toy image is 96×96 pixels, its centre is (48, 48), focal length is 48 pixels and height is 5 m. The camera is level and points straight down, with north at the top of the image. A target at pixel (60, 36) is 12 pixels east and 12 pixels north of the centre.

$$\Delta N = (48 - v) \times 5 / 48 ; \Delta E = (u - 48) \times 5 / 48$$

The target offset is therefore [1.25, 1.25] m. Its direction is correct, but its length exceeds the 0.5 m command limit. Scale the vector to [0.354, 0.354] m; do not clip each axis independently and accidentally allow a longer diagonal movement.

Run the sign checks

```
python -m unittest discover -s tests -v
```

The image conversion here is the ideal teaching-camera case. The full frame and timing conventions are on the preceding page.

Pass when: a marker north of the vehicle commands positive north, and a marker east commands positive east. These toy equations are only valid for the stated camera assumptions. For an inclined camera or changing yaw, use calibration and the full camera-to-world transform.

CHAPTER 1 SCOPE AND FOUNDATIONS

Visualise the image to ground conversion

The image and the ground use different axes. This diagram uses the ideal downward teaching camera so the sign convention can be checked by hand before introducing camera tilt, yaw or calibration error.

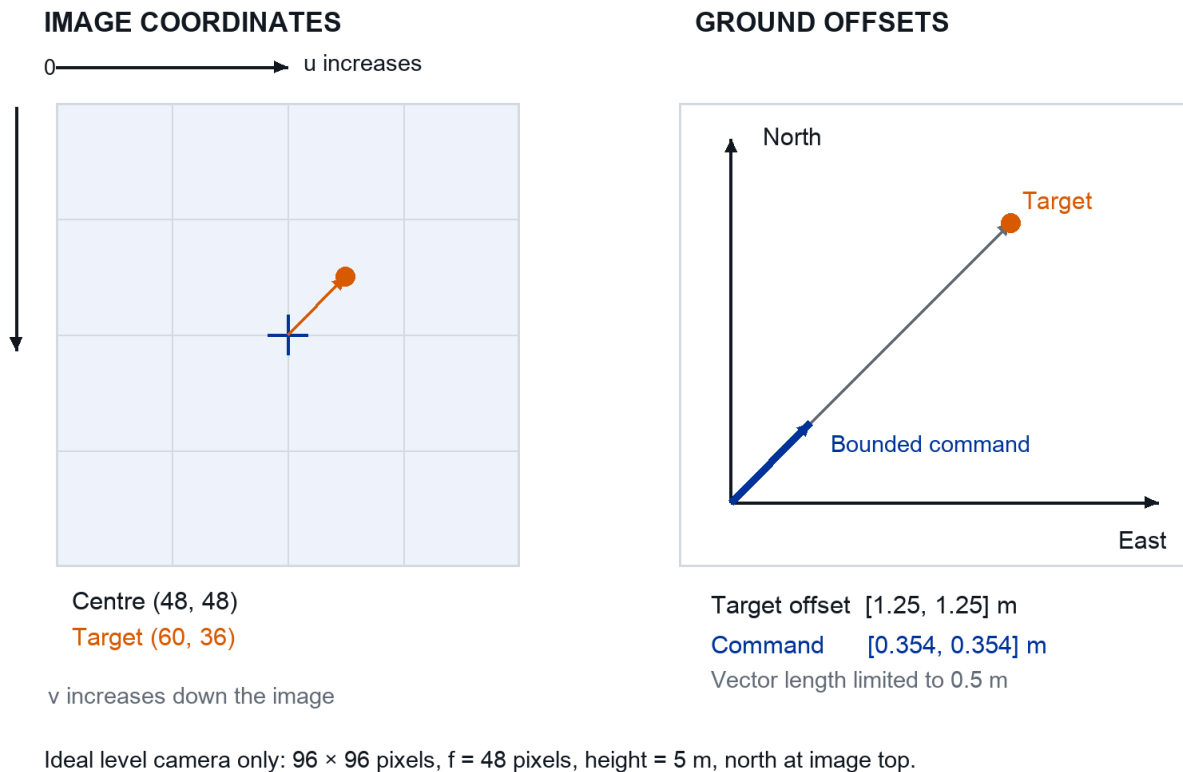


Figure 2. Worked geometry from the preceding example. Offsets are in north and east metres; the action vector is scaled to a 0.5 m length.

Follow the target across both panels

The target is right of and above the image centre. With north at the top, this means positive east and positive north. Both ground offsets are 1.25 m. The action keeps that direction but limits the length of the two-dimensional vector.

Check understanding

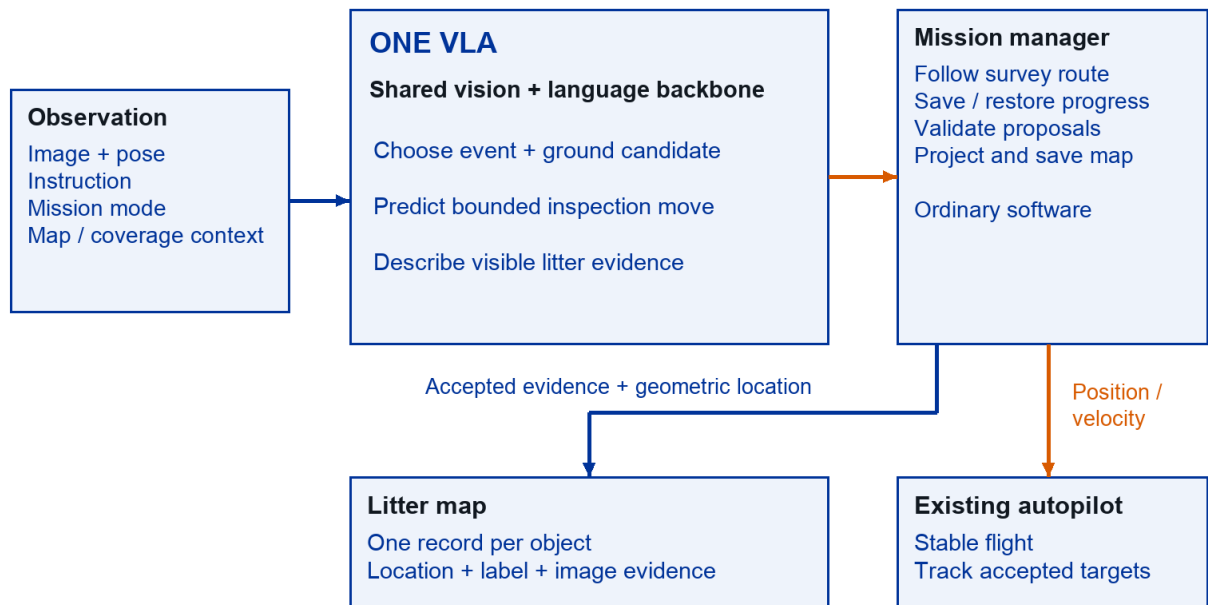
Move the target to pixel (36, 60). Predict both signs before running the simulator. Then explain why changing camera yaw would invalidate this simple conversion unless the image ray is rotated into the world frame.

CHAPTER 1 SCOPE AND FOUNDATIONS

One VLA decides and acts

The survey executor follows a predefined coverage path. One VLA monitors images and mission context, requests an inspection, predicts its movements, proposes a map record and requests resumption. The manager validates these requests and remembers the unfinished route.

ONE LEARNED MODEL ON THE ORIN



The manager supplies mission context. New observations close the loop after movement.

Figure 3. Proposed single VLA system. The model contains shared visual and language processing with several output heads. Mission memory, geometry and command checks are deterministic software; the flight controller remains separate.

Keep model output separate from accepted execution

Inputs: image and capture-time pose, instruction, mission mode, coverage progress, track summary, remaining budget and validity flags.

Outputs: event, grounded image region, semantic fields and optional bounded movement.

Execution: accepted event, accepted setpoint, rejection reason and new manager state are logged separately.

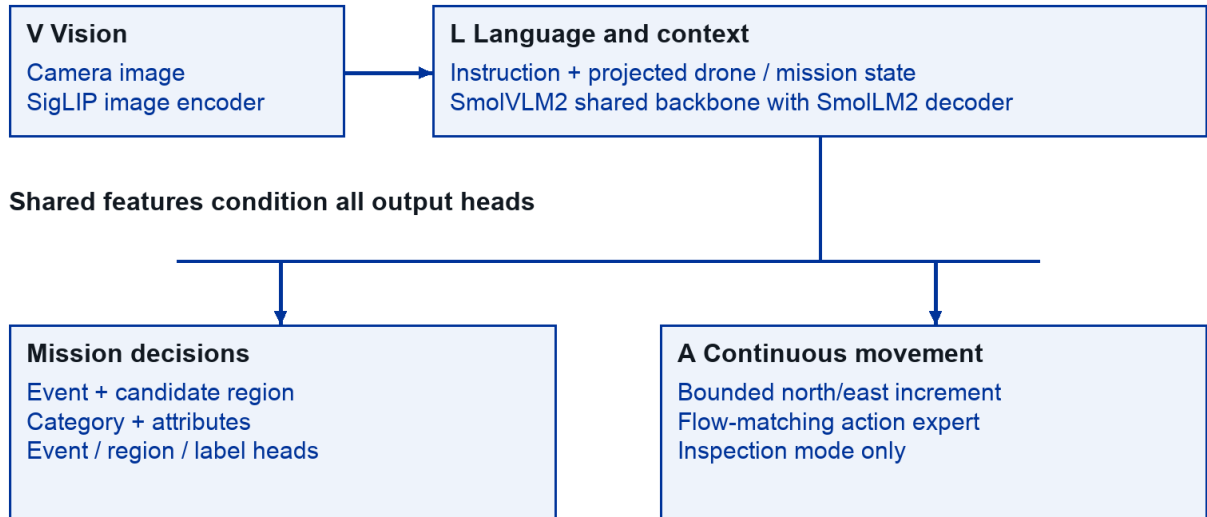
A complete target-system run must use the VLA's own candidate and semantic outputs. A separate detector may be used in the classical comparator or offline annotation, with that role recorded. The included SmoVLA scripts currently implement only the movement pilot.

CHAPTER 1 SCOPE AND FOUNDATIONS

Understand the V the L and the A

Start from the SmoVLA family: its visual and language features condition a flow-matching action expert [6, 23]. The mission heads below are proposed additions for this thesis, not functionality already supplied by the teaching code.

ONE VLA CHECKPOINT



Proposed additions: mission context, event, grounding and semantic heads.

Flow matching generates movement; it does not supply mission events automatically.

Ordinary software stores the route, projects locations and validates execution.

Figure 14. Proposed internal model. The visual and language backbone is shared. Discrete decisions and semantic labels use supervised heads; flow matching generates continuous actions. All heads belong to one deployed checkpoint.

Three meanings to keep distinct

V: the image encoder extracts visual features; the SmoVLA family uses a SigLIP encoder. **L:** the SmoVLM2 backbone combines visual features with instruction tokens; its language decoder is based on SmoLM2. Add projected drone and mission state to the selected implementation explicitly. **A:** the model predicts actions; flow matching is the training and sampling method for continuous movement.

The flow-matching vector field is defined over action samples, not directly over the physical field or air flow. An initial action is $[\text{delta_north}, \text{delta_east}]$ in metres. Altitude and yaw are fixed. A categorical event head is required for DETOUR, MARK and RESUME; these do not emerge automatically from a movement-only loss.

CHAPTER 2 LITERATURE AND RESEARCH POSITIONING

What the literature supports

The project connects four bodies of work: aerial litter detection, general robot policies, language-guided aerial control, and active perception. They support the proposed components, but they do not establish that the complete system will outperform a carefully tuned classical mission.

Detection and mapping already provide a baseline

Kraft et al. [1] demonstrate aerial litter detection and localisation, while Bartolo et al. [2] review datasets and UAV approaches. This makes a detector-based map an appropriate reference system. The research opportunity is to measure whether extra observations and learned control improve the cost or quality of that map. Existing imagery datasets do not replace flight demonstrations or surveyed test layouts.

Manipulation VLAs offer training ideas

RT-2 [4] represents robot actions as tokens; OpenVLA [5] provides an open pretrained policy; pi0 [18] and SmolVLA [6] explore continuous action generation. These works motivate adapting pretrained visual and language representations. Their manipulation results do not establish transfer to aerial dynamics, small litter or a new camera geometry. A drone action/state adapter and a controlled transfer study are necessary.

Aerial control supplies several comparisons

CognitiveDrone [7] studies a base aerial VLA and a variant with an additional reasoner. This thesis chooses the single-model route. See, Point, Fly [8] grounds visual waypoints and converts them into flight actions without policy fine-tuning. Together they motivate comparing a learned local policy with a strong perception-plus-controller baseline. NaVILA [13] offers another example of separating mid-level language actions from execution, but its evidence concerns legged robots.

Reasoning and active perception need outcome tests

ECOT [9] links reasoning to visually grounded robot features. DeepThinkVLA [10] examines the conditions under which reasoning benefits action. ReAct [11] and FlightGPT [12] inform the planner design. Their results motivate experiments; they do not show that an explanatory text field causes improvement on this field task.

The recent ESARBench [14], ActiveFly-Bench [15], ScoutVLA [16] and FLIGHT [17] address search, observation planning or language-guided control at different scales. Their tasks and success definitions differ from litter mapping. Use them to design memory, viewpoint and timing tests rather than comparing raw benchmark percentages.

A claim that the thesis can test

Suggested positioning: "We evaluate a single compact VLA for interrupting a survey, acquiring evidence and resuming coverage during aerial litter mapping, under measured mission and onboard compute budgets." A claim to be the first requires a separate, updated search; this guide does not establish priority.

CHAPTER 2 LITERATURE AND RESEARCH POSITIONING

Compare prior work by its evidence

Organise the related work around the decisions this thesis must make. Compare the task, action representation and evaluation domain. A reported success rate cannot be transferred between papers with different tasks, denominators or flight constraints.

Work	Relevant evidence or idea	Decision for this thesis
UAVVaste [1] and TACO [19]	Aerial litter localisation and contextual image annotations	Establish perception and mapping; collect separate action demonstrations
RT-2 [4] and OpenVLA [5]	Vision and language conditioned robot action prediction	Transfer representations while redefining drone state and action units
SmolVLA [6] and pi0 [18]	Continuous action generation from demonstrations	Use a native supervised action objective; benchmark one small candidate
CognitiveDrone [7]	Aerial action generation and an added reasoning module	Compare single-model decisions with the separate-reasoner alternative
See Point Fly [8]	Image waypoint grounding and conversion to flight displacement	Include a strong geometry or grounding based comparator
ECot [9] and ReAct [11]	Grounded intermediate decisions and observation dependent interaction	Evaluate the chosen task and its outcome, not text fluency

Translate a difference into an experiment

The proposed skill uses a downward camera, small ground objects and horizontal position increments. A pretrained manipulation policy has a different embodiment. An aerial navigation policy may use a forward camera and larger goals. These differences motivate controlled changes in image scale, geometry, action interface and observation delay.

State the contribution as a testable result

A suitable contribution is a reproducible comparison of rules and one unified VLA for survey interruption and litter inspection, with a documented dataset and measured resource and transfer limits. Establish novelty through an updated search; do not assume that adding a VLA to a drone is itself a new scientific claim.

CHAPTER 2 LITERATURE AND RESEARCH POSITIONING

Reasoning and active perception in aerial work

Active perception means moving to acquire evidence that is currently missing. For litter mapping, this could mean obtaining a sharper category view or reducing position uncertainty. That is a more specific goal than asking a model to produce a long explanation.

Read reasoning work with its training changes

ECoT [9] grounds intermediate reasoning in robot observations. DeepThinkVLA [10] studies reasoning in an action-model setting. Compare the supervision, architecture and compute used along with the reasoning format. FlightGPT [12] is an aerial vision and language navigation reference; NaVILA [13] concerns legged navigation and is an architectural comparison, not drone evidence.

Recent reference	Task emphasis	Relevant test to borrow
ESARBench [14]	Aerial search and rescue with memory and exploration	Does task memory reduce repeated search or missed regions
ActiveFly Bench [15]	Question answering, observation planning and language guided aerial control	Separate recognition, viewpoint choice and execution failures
ScoutVLA [16]	Evidence seeking viewpoint refinement for aerial question answering	Does another view resolve the specific uncertainty
FLIGHT [17]	Long-horizon aerial navigation with task-state reasoning and actions	Does the system keep the task consistent over successive decisions

DroneVLA [47] is a related aerial-manipulation concept, with a different task and component structure. These recent papers cover different tasks, datasets and evaluation settings. The cited 2026 records are preprints. Read their precise protocols and distinguish simulation, qualitative field demonstrations and quantitative real-world evaluation before comparing any numbers.

Test the decision head inside one VLA

For a controlled ablation, keep the unified model's perception and movement heads fixed. Compare its learned event head with a deterministic rule using the same visible records. Include inference delay and rejected events. Measure whether detours improve the map, and inspect unnecessary revisits. pi0.5 [44] provides an example of one model producing high-level and continuous actions; this is architectural precedent, not drone evidence.

A reasonable outcome is that the heuristic performs equally well. That result narrows where reasoning is useful and avoids attributing improvements to an explanation field that did not change the decision.

CHAPTER 2 LITERATURE AND RESEARCH POSITIONING

Read these six papers first

For each paper, skim the abstract, system figure and limitations first. Read the implementation and evaluation closely only when you reach its corresponding build stage. Keep a half-page note using the paper-note template at the end of this guide.

1 UAVWaste and the classical application

Read [1], Kraft et al., 2021. Take the detection-to-map pipeline as your starting reference. Extract the camera assumptions, detection evaluation and localisation approach. Its application evidence does not answer whether a learned inspection policy is needed. **Produce:** a drawing of your baseline and a list of differences from the paper.

2 The aerial litter review

Read [2], Bartolo et al., 2026. Use the review to compare the aerial viewpoint, class definitions and annotation types of available datasets. Do not treat a broad material taxonomy as visually inferable in every image. **Produce:** a dataset choice with one reason to use it and one limitation for your field.

3 OpenVLA

Read [5], Kim et al., 2024. Understand how images and instructions become robot actions and what fine-tuning changes. Its reported evaluations are on manipulation, so they cannot be carried over to drones. **Produce:** your proposed observation, language and action tensors, with units and dimensions.

4 SmolVLA

Read [6], Shukor et al., 2025, with [23]. Focus on the state input, continuous action chunks, training data and inference/execution separation. Consumer hardware accessibility is motivation, not a drone-latency guarantee. **Produce:** one inspected data batch, an adapted configuration and a small inference profile.

5 CognitiveDrone

Read [7], Lykov et al., 2025. Examine its aerial action representation and the role of the separate reasoner. Its simulated cognitive tasks differ from small-object inspection. **Produce:** one useful architectural idea and one experiment needed before transferring that idea to this project.

6 One model for decisions and actions

Read [44], Physical Intelligence et al., 2025. Study the distinction between shared representations, high-level prediction and continuous action generation. **Produce:** a list of mission labels and losses needed beyond ordinary SmolVLA movement training. Keep See, Point, Fly [8] as an additional grounding-based comparator.

Reading checkpoint: you can explain which component is learned, where language changes behaviour, what the autopilot controls and which experiment would show a benefit. If any answer is unclear, revisit the system diagram before reading more papers.

CHAPTER 2 LITERATURE AND RESEARCH POSITIONING

Read further when the experiment needs it

For the action representation

Use RT-2 [4] for tokenised actions, pi0 [18] for flow-based action generation and NaVILA [13] for mid-level language actions. Compare the representation, feedback interval and execution layer. Sa et al. [3] provide a broader survey. These sources are background; implementing all model families would dilute the core experiment.

For the reasoning comparison

Read ECoT [9] and DeepThinkVLA [10] together: record what supervision or architecture changes accompany reasoning. Read ReAct [11] for interaction with tools and FlightGPT [12] for a trained aerial navigation approach. Ask which gains could come from extra data, compute or action structure rather than the presence of a rationale.

For choosing where to look next

ESARBench [14] studies embodied aerial search with memory. ActiveFly-Bench [15] distinguishes answering, observation planning and control. ScoutVLA [16] focuses on acquiring missing evidence through viewpoint changes. FLIGHT [17] separates slower task-state reasoning from continuous action generation. They motivate useful extensions; their larger task settings are not required implementation targets.

For perception and geometry

Use TACO [19] for contextual litter annotations. Grounding DINO [20] and Qwen2.5-VL [21] are references for comparators or offline annotation; they are not additional onboard models in the chosen system. Cabreira et al. [22] provide coverage-planning context. Consult OpenCV [28] when implementing calibration and projection, rather than reconstructing geometry from prompts.

Maintain a focused review

This is a selected reading guide, with linked publication records checked during this revision. To turn it into the thesis literature review, search the following combinations and log the date, query, source and inclusion decision:

- ("UAV" OR "drone" OR "aerial") AND ("vision language action" OR "VLA" OR "vision language navigation").
- ("litter" OR "trash" OR "waste") AND ("UAV" OR "aerial") AND ("detection" OR "mapping" OR "active perception").
- ("reasoning" OR "active perception") AND ("inspection" OR "object search") AND ("drone" OR "robot").

Follow references backward and citations forward from the six core papers. Include papers with a relevant task, explicit action/perception interface and evaluable evidence. Record simulation versus real experiments, code/data availability and publication status. Refresh near submission and distinguish preprint versions from peer-reviewed records. Do not infer absence of prior work from a single search.

CHAPTER 3 ACTION CONTRACTS AND FIRST LEARNING LOOP

Your first two weeks

The schedule assumes basic Python. If arrays, functions, virtual environments or reading JSON are unfamiliar, spend the first three sessions on those topics and move the dates. The order matters more than the calendar.

Session	Work	Evidence to bring
1	Install the laptop environment and run the tests	Test output and Python version
2	Run the first baseline and inspect every saved file	Replay plus an explanation of one command
3	Draw the coordinate axes and change the requested category	Three trajectories from the same seed
4	Generate and validate demonstrations	Split counts and one annotated record
5	Train the tiny model	Training history and selected checkpoint
6	Evaluate once on the held-out scenes	Baseline and learned success table
7	Run language interventions and inspect failures	Two failure explanations with images
8	Meet the lab engineer and start default PX4 SITL	Version record and default simulator screenshot
9	Generate the field and verify the camera image	Labelled north and east marker checks
10	Run the simulated square and review the bridge contract	Telemetry plot and a list of remaining integration issues

Prepare only the concepts you need

Before session 3, learn vectors, a coordinate frame and metres versus pixels. Before session 5, learn a training set, a validation set, a loss and an optimiser. Before session 8, learn what the autopilot, simulator and companion computer each do. You do not need to derive MPC or quaternion calculus to run the first exercise.

Checkpoint: explain the whole laptop loop aloud in five minutes. If you can only recite commands, repeat one episode with different inputs and predict its behaviour before running it.

CHAPTER 3 ACTION CONTRACTS AND FIRST LEARNING LOOP

Use a local position command contract

The recommended first VLA outputs a **two-dimensional local position increment** at fixed inspection altitude and yaw. The flight adapter converts this to a position setpoint for PX4. Position holding tolerates a modest policy update rate and makes a useful starting point for the Orin.

Define the two learned action values

```
action = [delta_north_m, delta_east_m]
anchor = estimated_position_at_observation_capture
setpoint_north = anchor.north + delta_north_m
setpoint_east  = anchor.east  + delta_east_m
setpoint_down  = fixed_inspection_down
setpoint_yaw   = fixed_inspection_yaw
```

Convert the increment to an absolute local setpoint **once per accepted prediction**. Repeated publication must resend the same setpoint, not repeatedly add the increment. Limit the increment norm and keep the target inside a local inspection envelope. Include measured velocity, height and orientation in the observation so the policy can account for momentum and view geometry.

A proposed initial envelope

In simulation, start with at most 0.5 m per increment, a 2 Hz policy target, fixed altitude and fixed yaw. These are settings to validate, not guaranteed device performance or flight limits. Cap the autopilot's permitted horizontal motion using the lab's established configuration. A position jump bound alone is not a speed bound.

Every prediction needs metadata

```
task_id, observation_id, frame="local_NED"
anchor_position_m, delta_position_m
image_capture_time, generated_time, valid_until
model_version, action_schema_version
```

Reject nonfinite outputs, unknown frames, out-of-envelope targets and actions for old task IDs. Expiry is based on observation age as well as generation time: a fresh answer computed from an old frame can still be unusable. Convert units explicitly at the SDK boundary; API names and yaw units depend on the chosen binding.

Completion and stopping

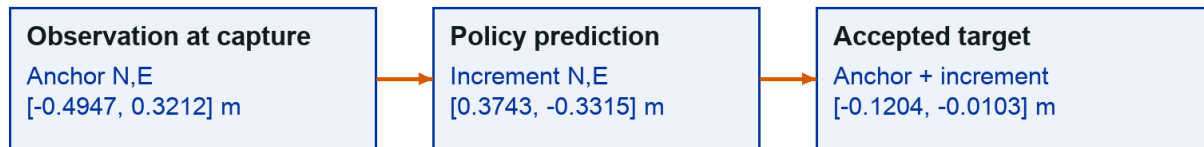
Use a separate rule to detect stable centring and save the inspection. The policy may output zero, but zero displacement anchored to an old pose is not a reliable stop signal. On expiry, the executor requests a tested hold at the current valid position or the configured fallback if position is invalid. Takeoff, landing and return remain mission states.

CHAPTER 3 ACTION CONTRACTS AND FIRST LEARNING LOOP

See why a command must keep its anchor

A model predicts an increment relative to a particular observation. The executor converts that increment to an absolute target once. Repeating the same target lets the autopilot track it while the next observation is processed.

ONE PREDICTION CREATES ONE ABSOLUTE TARGET



WALL CLOCK EXAMPLE

Illustrative times; measure your own latency.



Send the SAME absolute target while it remains valid.

Do not add the increment again on every resend.

Reject stale observations or expired commands; use the tested hold response.

Figure 4. Actual seed 7 position values with a schematic wall-clock timeline. The illustrated delays are not measured hardware timings.

Read the two kinds of time

Capture time identifies the pose that belongs to the image. The later send time describes when the command reached the execution path. Log both, and measure their difference. A newly generated answer can still be based on an old image.

A useful failure exercise

Keep one prediction fixed and resend it several times in simulation. The target coordinates should remain constant. If the target walks away from the vehicle, the adapter is adding the increment repeatedly.

CHAPTER 3 ACTION CONTRACTS AND FIRST LEARNING LOOP

Velocity commands and the independent watchdog

A velocity policy is a valid alternative when the existing drone API or lab practice favours it. Treat it as a separate action schema and experiment. Do not reinterpret a position-trained checkpoint as a velocity policy.

A simple horizontal velocity contract

```
action = [v_north_m_s, v_east_m_s]
v_down = 0; yaw = fixed_inspection_yaw
# SDK adapter transmits this in the documented frame.
# Vertical-velocity hold behaviour must be verified.
```

Limit speed, acceleration between accepted commands and the inspection region. A zero vertical velocity request does not express an absolute height target; test drift and the actual autopilot mode semantics. If body-frame velocities are used, store the frame in both dataset and command and rotate with the measured yaw. Never omit the frame because the vector has only two values.

A model loop is not the command loop

```
model worker: read newest observation -> predict -> mailbox
executor, e.g. 20 Hz:
    check vehicle state, task ID and observation age
    if newest action is valid: validate and send
    else: execute the tested hold or fallback
logger: save proposal, accepted command and reason
```

The executor runs in an independent process or suitably isolated runtime. An SDK may automatically resend the last setpoint; this keeps offboard alive even after inference freezes [24]. A watchdog must override the stale command. Test a model crash, GPU stall, dropped camera stream and a delayed mission-event prediction in SITL.

Choose deadlines from motion measurements

Measure capture-to-command delay and the stopping response at the selected speed. Set the allowed observation age and geofence margin so the vehicle can remain inside the envelope after the worst accepted delay plus stopping distance and position uncertainty. Do not increase the deadline merely to hide missed inference budgets.

Position or velocity for the thesis

Use position increments for the initial learned skill. Keep velocity support in the adapter if the platform needs it. Run the same selected contract for the expert, dataset, model and baseline. This preserves the scientific question about visual decisions while leaving the autopilot's control implementation unchanged.

CHAPTER 3 ACTION CONTRACTS AND FIRST LEARNING LOOP

Lab 1 Run your first episode

Goal: see the complete observation-to-command loop before installing a flight simulator. Unzip the starter kit and open a terminal in its root directory. These commands use macOS or Linux; the README gives the Windows activation command.

```
python3 -m venv .venv
source .venv/bin/activate
python -m pip install -r requirements.txt
python -m unittest discover -s tests -v
python -m litterlab demo --target can --out runs/first
```

Expected output: 10 passing tests. With the default seed 7, the classical baseline reports success after 9 decisions and 4.5 simulated seconds. Open the replay and trajectory files in runs/first.

File	What to inspect
replay.gif	The can approaches the centre crosshair as the vehicle moves
trajectory.png	North is up and east is right; the trace approaches the can
steps.jsonl	One image-time observation and proposed action per row
map_local.json	Local object coordinates and evidence timestamps
result.json	Success, step count, seed and policy name

Change one variable

```
python -m litterlab demo --target bottle --out runs/bottle
python -m litterlab demo --target paper --out runs/paper
```

Use the same seed so the field and initial position stay the same. Predict the direction of the first command for each category. The toy language parser recognises one category word; this exercise does not yet learn natural language.

If it fails

“No module named litterlab” usually means you are outside the kit root. “File exists” means the output directory already contains a run; choose a new name. Missing NumPy or Pillow usually means the wrong Python environment is active.

Bring to supervision: the three trajectories and a short explanation of why the requested target changed the motion.

CHAPTER 3 ACTION CONTRACTS AND FIRST LEARNING LOOP

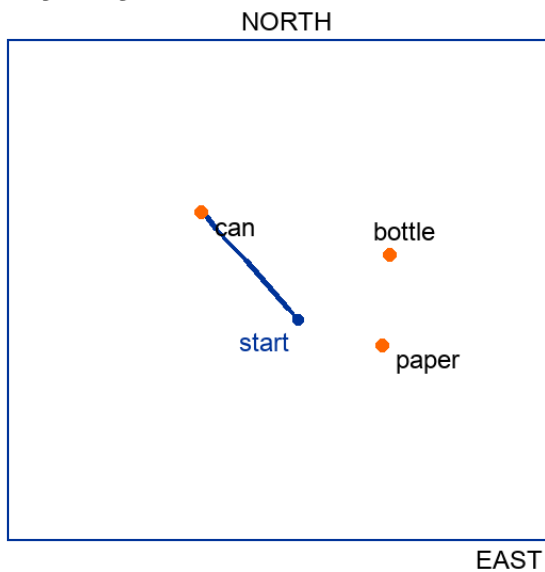
Lab 2 Read one command as evidence

Open `runs/first/steps.jsonl`. This is the first row, rounded for readability; the saved file retains full precision.

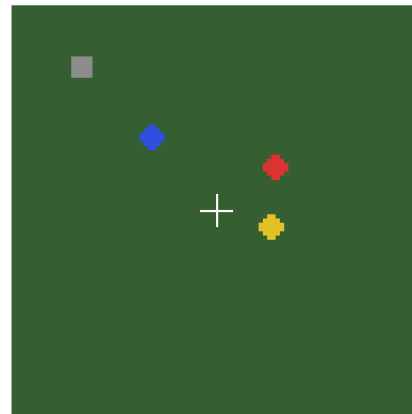
```
capture_time_s: 0.0
instruction: "centre above the can"
position_ned_m: [-0.4947, 0.3212, -5.0]
velocity_ned_m_s: [0.0, 0.0, 0.0]
action_delta_ne_m: [0.3743, -0.3315]
accepted_position_ne_m: [-0.1204, -0.0103]
```

The action asks to move north and west. Add it to the capture position: $-0.4947 + 0.3743$ gives -0.1204 north, and $0.3212 - 0.3315$ gives -0.0103 east. Its vector length is 0.5 m. The absolute target is fixed while the toy vehicle moves towards it.

Trajectory in local coordinates



First camera frame



Task centre above the can
Crosshair marks image centre

Figure 5. Actual seed 7 trajectory and first camera frame. Orange map points show evaluator truth; the classical policy uses the image. Blue is the can, red the bottle and yellow the paper. This is a teaching simulation.

Inspect the code in this order

In `litterlab/core.py`, read `observe()`, `classical()`, `bounded()` and `apply()`. In `__main__.py`, read `rollout()`. The `expert()` function may read object truth to make demonstrations. An observation passed to a policy must not contain those hidden object coordinates.

Pass when: you can compute the accepted target by hand and explain why the vehicle's next measured position can differ from that target.

CHAPTER 3 ACTION CONTRACTS AND FIRST LEARNING LOOP

Lab 3 Make and inspect demonstrations

An episode is one trial from reset to success or timeout. It contains a sequence of observations and actions. The word does not imply reinforcement learning. Here a scripted expert provides action labels, so you are collecting data for supervised imitation learning.

```
python -m litterlab collect --out runs/data
python -m integration.data --data runs/data
```

Expected output: 210 episodes with no expert failures in the supplied setup. There are 150 training, 30 validation and 30 test episodes, containing 1,144, 210 and 215 frames respectively. Distinct scene-seed ranges keep layouts separate between splits.

Saved item	Purpose	May the policy read it
images and steps.jsonl	RGB, task, state, label and accepted target	Yes, through the defined observation fields
manifest.json	Episode identity, seed, split and acquisition schema	For loading and reproducibility
samples.npz	Compact features used by the tiny teaching learner	Yes for this exercise
evaluator_only.json	True object locations for scoring	No

Read an episode before trusting it

Open train_0000. Match its first image to the instruction and motion direction. Confirm that capture times increase by 0.5 s, the state has seven values and the action has two. The validator checks finite values, action limits, image dimensions, timestamps and duplicate scene seeds.

Understand this dataset's limits

The objects have fixed artificial colours. The task vocabulary has three categories. Targets are nearby, the ground is flat and the camera is ideal. This dataset teaches interfaces and learning mechanics. It cannot establish litter recognition, long-range search, obstacle avoidance or transfer to outdoor imagery.

Pass when: you can explain which information the expert can see and which information the deployed policy will actually receive.

CHAPTER 3 ACTION CONTRACTS AND FIRST LEARNING LOOP

Episodes do not mean reinforcement learning

An **episode** is one trial, from reset to completion or failure. It can be recorded by a human, a scripted expert or a learned policy. The word describes the data structure, not the training method.

Method	What updates the model	Role in this thesis
Supervised perception	Image labels such as boxes and classes.	Train shared grounding and semantic heads.
Behaviour cloning or imitation	Expert actions paired with observations and instructions.	Recommended first VLA training method.
DAGger style collection	Expert labels on states visited by the learner, then supervised retraining.	Optional simulation remedy for compounding errors [33].
Reinforcement learning	A reward and interaction experience used to optimise a policy.	Optional extension; not required by episodes.
Supervised mission decisions	Events paired with visible evidence and mission context.	Train the unified VLA event head.

One concrete imitation example

At time t the image shows a can to the east of the image centre, the instruction requests the can, and the expert commands a bounded eastward position increment. This observation/action pair is a supervised training example. The next image and action form the next step of the same episode. Success labels are useful for evaluation but do not turn this into RL.

Understand the loss used by the model

A simple regression policy can minimise error between predicted and expert actions. SmolVLA uses a flow-matching action objective [6, 39]: it learns to transform noisy action samples towards expert action sequences, conditioned on the observation. That remains supervised imitation. Use the selected model's native training objective and correct masks rather than assuming every VLA uses plain action MSE.

Why closed-loop tests are essential

A small prediction error moves the drone to a slightly different view. Further errors can accumulate because that view was rare in the expert data. A low validation loss is therefore not sufficient. Run the learned policy in the simulator, inspect visited states, and collect valid corrective demonstrations for systematic failures.

When RL would be justified

Consider RL only after imitation is working and there is a bounded question that demonstrations cannot answer, such as trading inspection duration against task success in simulation. It would require reward design, exploration constraints, additional baselines and a separate evaluation. Do not begin with online real-drone RL or add it merely because the simulator exposes step and reset functions.

CHAPTER 3 ACTION CONTRACTS AND FIRST LEARNING LOOP

Lab 4 Train your first policy

The tiny policy receives all three colour-centroid measurements, their visibility flags, the requested category and horizontal velocity. It predicts the expert's two-dimensional position increment. It is a small multilayer perceptron implemented in NumPy. It is not the pretrained VLA proposed for the thesis.

```
python -m litterlab train --data runs/data --out runs/tiny
```

The default run uses 200 epochs, batches of up to 128 samples, Adam with learning rate 0.002 and seed 0. One epoch visits the training data once. One optimiser step updates the model on a batch; the supplied run made 1,800 such updates.

What the loss measures

Divide each action component by the fixed 0.5 m scale. The mean squared error compares predicted and expert values across the two components. A loss of zero means exact agreement on those samples. It does not mean that a complete mission succeeds.

$$L = (1 / 2B) \sum || \hat{a} - a ||^2$$

Training chooses model parameters using training samples. Validation chooses the saved checkpoint. The supplied run's best validation MSE was 0.0223172 in normalised units. Expect small platform-dependent numeric differences. The test set is not used for checkpoint selection.

Inspect the saved result

runs/tiny contains policy.npz, history.json and config.json. Plot training and validation losses against epoch, or inspect their trend in the JSON. Falling training loss with rising validation loss suggests overfitting; equally poor losses suggest an input, label or capacity problem.

Predict a failure before testing

The expert labels describe expert trajectories. A learned policy may drift into states rarely visited by the expert. Consequently, low offline action error can coexist with poor closed-loop success. Write this prediction in the notebook before the next lab.

Pass when: you can explain the difference between an episode, an epoch, a batch, an optimiser step and an evaluation trial without treating them as synonyms.

CHAPTER 3 ACTION CONTRACTS AND FIRST LEARNING LOOP

Lab 5 Evaluate the behaviour

```
python -m litterlab evaluate --out runs/classical
python -m litterlab evaluate --model runs/tiny/policy.npz \
  --out runs/learned
python -m litterlab evaluate --model runs/tiny/policy.npz \
  --language swapped --out runs/swapped
python -m litterlab evaluate --model runs/tiny/policy.npz \
  --language empty --out runs/empty
```

Success requires remaining within 0.20 m of the requested object with speed below 0.18 m/s for two consecutive decision ticks, within 30 simulated seconds. Every condition uses the same 30 held-out scenes and original requested goals.

Condition	Success	Mean final error
Classical colour baseline	30 / 30	0.085 m
Tiny learned policy with normal task	24 / 30	0.182 m
Tiny policy given a different task	0 / 30	2.127 m
Tiny policy with task input removed	0 / 30	1.462 m

These are measured teaching-simulator results, not projections. In the swapped condition, the policy receives a different category while the evaluator retains the original goal. The drop shows task-input sensitivity. Because the task is a one-hot category, it does not establish general language understanding or reasoning.

Investigate before adding complexity

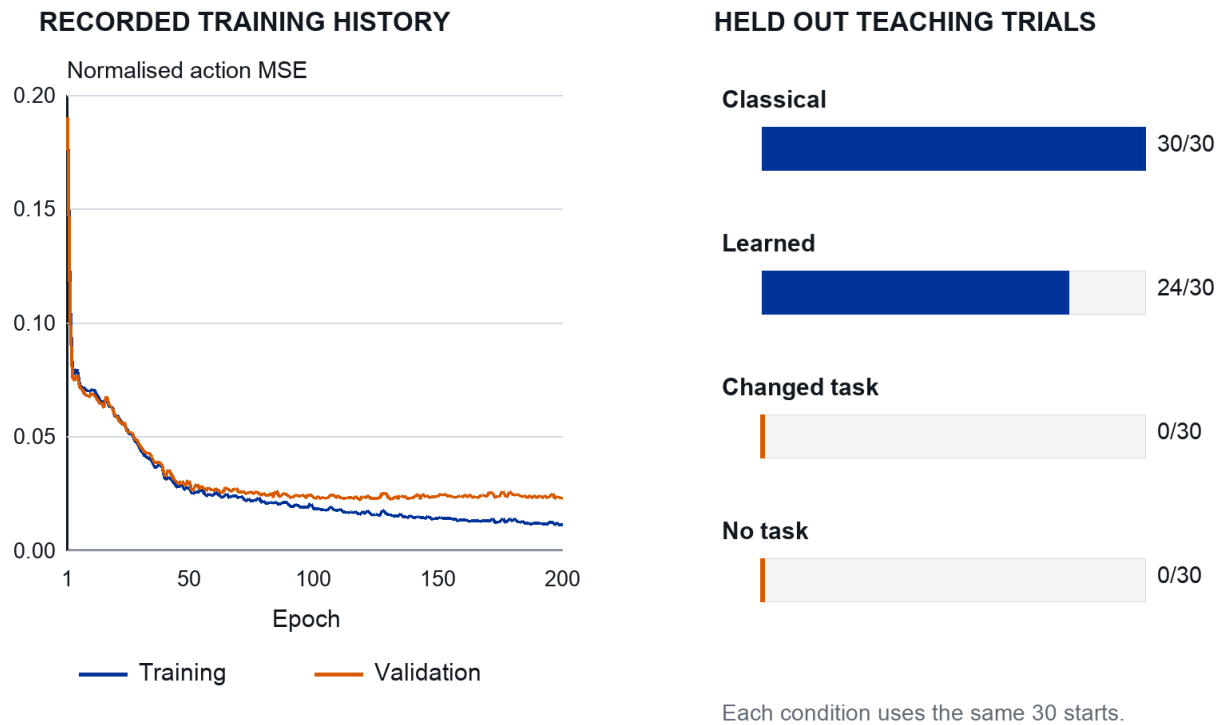
Open trials.json and identify a failed seed. Replay it with demo --model runs/tiny/policy.npz --seed SEED --target CATEGORY and a new output folder. Decide whether it oscillates, stops too far away or chooses the wrong object. Do not tune against these test failures and then present the same scenes as an untouched test.

Pass when: your report states that the learned baseline is weaker here and explains one failure using its image, command and trajectory. Keep the classical baseline in every later comparison.

CHAPTER 3 ACTION CONTRACTS AND FIRST LEARNING LOOP

Read the measured teaching results

A learning curve and a behaviour test answer different questions. The curve measures agreement with demonstration actions. The bars measure whether the closed-loop policy completed the requested task.



Source: `bundled examples/tiny_model/history.json` and `examples/eval_*/summary.json`.

Figure 6. Recorded NumPy teaching run. Curves show 200 training epochs; bars show successes out of the same 30 held-out starts. These are not SmolVLA or real-flight results.

What this example establishes

The tiny learner improves its action loss but succeeds in 24 of 30 trials, compared with 30 of 30 for the classical baseline. The task intervention bars score against the original goal. They show input sensitivity, not general language understanding.

Reuse the plotting pattern

Save the raw history and trial records for your own experiments. Plot both action loss and closed-loop outcomes, then show representative failures. These curves come from one training seed and do not quantify variability across training runs.

CHAPTER 4 FLIGHT SIMULATION AND THE CLASSICAL MISSION

Lab 6 Start the flight simulator

The laptop simulator taught the software contract. PX4 software in the loop, or SITL, adds the actual autopilot running against a simulated vehicle. Gazebo supplies the scene, sensors and vehicle physics. We do not write a new stabilising controller.

Use a lab Ubuntu 22.04 workstation with PX4 v1.16.0 and Gazebo Harmonic as the candidate pinned stack. Follow the matching PX4 setup documentation and save the final versions. This combination and the supplied integration scripts still need the lab smoke test. [29, 30]

```
git clone --recursive --branch v1.16.0 \
  https://github.com/PX4/PX4-Autopilot.git
cd PX4-Autopilot
bash Tools/setup/ubuntu.sh
make px4_sitl gz_x500
```

The setup script installs system dependencies. Do this with the lab engineer on the designated machine. Keep this environment separate from the laptop NumPy environment and the workstation's VLA training environment.

Record what works before changing the world

Save the PX4 commit, submodule commits, OS release, Gazebo version and launch command. Confirm that the default x500 appears, sensors publish, the autopilot becomes ready and the simulator can be closed and restarted. Keep the PX4 console log.

Use the supplied files next

integration/prepare_gazebo.py copies the installed checkout's default world and generates a separate model with a downward camera. integration/record_camera.py records camera frames. integration/px4_square.py sends position targets. The launch instructions on page 31 give the multi-terminal commands and resource paths.

Pass when: the default simulator works from a fresh terminal using the recorded instructions. If the default world fails, fix the installation before debugging your custom world or model.

Boundary: syntax checks performed for this package do not establish that these scripts fly a simulated or real aircraft. The first successful lab run becomes the project's supported environment.

CHAPTER 4 FLIGHT SIMULATION AND THE CLASSICAL MISSION

Lab 7 Add a field and camera

From the starter-kit root, set LITTER_PX4 to the actual checkout location and generate new assets. The next page gives the full launch command with GZ_SIM_RESOURCE_PATH. The generator leaves the PX4 checkout unchanged.

```
export LITTER_PX4="$HOME/PX4-Autopilot"
python -m integration.prepare_gazebo \
  --px4 "$LITTER_PX4" --out runs/gazebo
```

The generated world has three static coloured blocks with known locations. The model includes the checkout's x500 plus a camera at 10 Hz, 640 × 640 RGB and a 90-degree horizontal field of view. These oversized targets are for geometry checks; replace their sizes and appearance before collecting research data.

Complete the launch commands on the next page before running the camera checks below.

Capture images and verify the axes

```
gz topic -l
gz topic -i -t /litter_camera/image
python3 -m integration.record_camera \
  --out runs/camera --seconds 10
```

The recorder needs the Harmonic Python bindings in its active environment. It saves frame files and simulation timestamps. It does not yet associate a pose with each image. Its expected image format is tightly packed RGB; a different format should fail visibly, not be silently misread.

Four checks before using images for commands

- Hover over a large marker. Confirm that it is centred at the expected vehicle position.
- Move the marker north, then east. Verify the image directions at yaw zero and again after a yaw change.
- Change height. Measure how the marker's pixel width changes; compare with focal length divided by height.
- Stop the vehicle and capture a burst. Confirm increasing timestamps, no blank frames and stable marker coordinates.

Pass when: you have a labelled image and a written camera-to-body convention. Verify coordinate signs and extrinsics even when the camera image looks plausible. [40]

CHAPTER 4 FLIGHT SIMULATION AND THE CLASSICAL MISSION

Launch the generated world and capture images

Use these commands after the default x500 simulator has passed Lab 6 and the assets have been generated in Lab 7. Close any running Gazebo instance before changing the world. Set the two paths to your actual directories. The commands are supplied integration instructions; they have not been executed in this authoring environment. [30, 31]

```
export LITTER_KIT="$HOME/VLA_Drone_Starter_Kit"
export LITTER_PX4="$HOME/PX4-Autopilot"
export LITTER_ASSETS="$LITTER_KIT/runs/gazebo"
export LITTER_GZ="$LITTER_PX4/Tools/simulation/gz"
export GZ_SIM_RESOURCE_PATH="$LITTER_ASSETS/models"
export GZ_SIM_RESOURCE_PATH="$GZ_SIM_RESOURCE_PATH:$LITTER_ASSETS/worlds"
export GZ_SIM_RESOURCE_PATH="$GZ_SIM_RESOURCE_PATH:$LITTER_GZ/models"
export GZ_SIM_RESOURCE_PATH="$GZ_SIM_RESOURCE_PATH:$LITTER_GZ/worlds"
cd "$LITTER_PX4"
PX4_SYS_AUTOSTART=4001 PX4_SIM_MODEL=gz_x500_litter \
  PX4_GZ_WORLD=litter ./build/px4_sitl_default/bin/px4
```

This example sets a task-specific resource path for the current terminal. If the lab requires additional resource directories, append them explicitly. The generated model includes the checkout's x500 and preserves its existing flight behaviour.

Record the camera in a second terminal

Use the Python environment containing Gazebo Harmonic's Transport 13 and Messages 10 bindings, plus Pillow. The recorder uses these imports:

```
from gz.transport13 import Node
from gz.msgs10.image_pb2 import Image

cd "$LITTER_KIT"
gz topic -l
gz topic -i -t /litter_camera/image
python3 -m integration.record_camera \
  --out runs/camera --seconds 10
```

Define LITTER_KIT again in the second terminal if needed. Expect 640 × 640 RGB frames and increasing simulation timestamps. If no frames arrive, inspect resource lookup, the world Sensors plugin and render errors. Confirm the image axes before using any pixels for movement or mapping.

Continue with Lab 8, then the synchronised bridge review on page 33. Camera recording alone does not associate capture-time pose with each image.

CHAPTER 4 FLIGHT SIMULATION AND THE CLASSICAL MISSION

Lab 8 Send position commands

Run only against the isolated software simulator for this exercise. No physical aircraft should be connected. The supplied script uses the MAVSDK 2.8.4 API pin, listens locally, waits for position readiness and commands a small square. [43]

```
python -m pip install mavsdk==2.8.4
python -m integration.px4_square --sitl --out runs/square
```

It arms the simulated vehicle, requests 5 m above local origin, visits the corners of a 2 m square, returns to the start and commands landing. Each waypoint needs less than 0.25 m position error and less than 0.2 m/s speed for one second. A waypoint times out after 30 seconds.

Inspect the telemetry rather than just the animation

Plot measured north and east against commanded north and east from position.jsonl. Measure settling time, overshoot and steady error. Preserve the PX4 log. The script records host receive times; these are useful for the exercise but are not sufficient to align a moving camera with capture-time vehicle pose.

An independent supervisor is the next component

A model process can hang while MAVSDK continues sending its previous target. Offboard-message streaming therefore does not prove that the model is healthy. A separate supervisor must enforce command age, observation age, the field boundary, altitude and speed limits, plus a reviewed hold or landing response.

In SITL, kill inference, freeze image timestamps, insert a non-finite action, replay an old command and interrupt communications. Check the logged rejection reason and the actual vehicle response. PX4's offboard loss configuration must be reviewed on the selected version; it is not replaced by a Python try/finally block.

Pass when: the square completes and the telemetry is explained. Complete the fault tests with the bridge and independent supervisor on the next page. The supplied square script is a flight-interface exercise, not the full watchdog or an approved real-flight program.

CHAPTER 4 FLIGHT SIMULATION AND THE CLASSICAL MISSION

Build the synchronised bridge with the lab

This is the main remaining engineering step between the supplied exercises and closed-loop flight simulation. Do it jointly with the lab engineer. The kit deliberately keeps the unverified bridge visible instead of pretending that a camera recorder plus a square script already forms a VLA drone.

Agree the observation message

```
Observation
  image_id, image_capture_time, clock_domain
  pose_at_capture, velocity_at_capture, attitude_at_capture
  camera_intrinsics, camera_to_body_transform, task_id
Command
  observation_id, task_id, delta_NE_m, capture_anchor_NE_m
  computed_time, expires_time, accepted_target_NE_m
```

Synchronise and anchor

Use the PX4 offboard and MAVSDK interface references [25, 32]. Choose one flight bridge with access to PX4 time synchronisation and timestamped state, and document how simulator time, PX4 time and host time relate. Buffer state messages. Interpolate position to the image timestamp and interpolate attitude with the appropriate rotation representation. Reject images outside the valid state buffer or with an excessive time mismatch.

For an initial stationary capture experiment, hold the vehicle and verify timing before adding motion. Do not use “nearest message received by Python” as an unexplained substitute for capture-time pose. Keep simulator truth for the evaluator; the policy should receive the estimator stream intended for deployment.

Use an explicit supervisor loop

```
at each supervisor tick:
  if observation stale or task changed: enter reviewed hold
  elif new valid proposal: anchor once and check limits
  elif accepted command expired: enter reviewed hold
  transmit the latest accepted absolute target
  log observation, proposal, acceptance and actual state
```

Start with a 0.5 s command-validity target and a 20 Hz supervisor; these are engineering hypotheses to validate against measured latency and braking distance. Give every proposal an observation ID so an old result cannot overwrite a newer task.

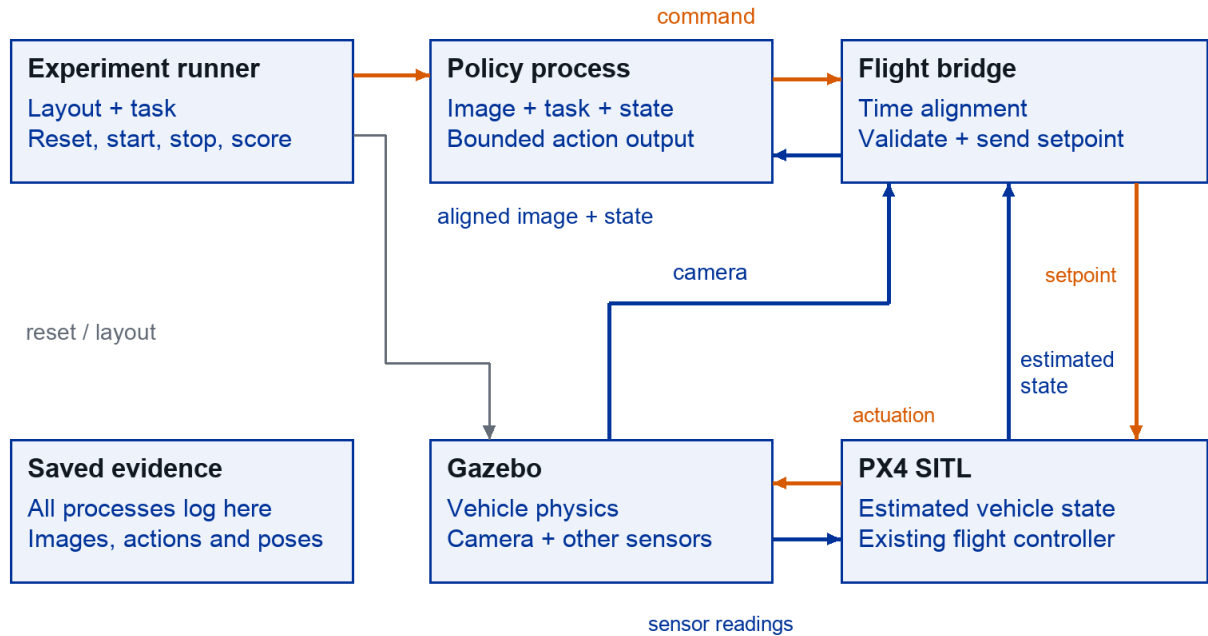
Pass when: a moving-marker test stays geometrically correct and every stale, repeated or invalid proposal is handled as specified. Record the bridge implementation, measured time error and fault-test log before connecting SmolVLA.

CHAPTER 4 FLIGHT SIMULATION AND THE CLASSICAL MISSION

Trace the simulator interfaces

The renderer, autopilot and learned policy are different programs. Draw their messages explicitly so you can locate failures in timing, coordinates, command execution or image delivery.

THE FLIGHT SIMULATION LOOP



The bridge pairs the camera with PX4 estimated state before delivering an observation.

Privileged ground truth goes only to the evaluator or expert, never to the deployed policy.

Figure 7. Proposed flight-simulation wiring. The supplied recorder and square script are pieces of this loop; the synchronised bridge still needs lab integration.

Build the loop in stages

First run Gazebo and PX4 with a scripted position command. Then add a timestamped camera observation and estimated state. Test the classical policy through the same bridge before connecting a learned model.

Keep evaluation information separate

The experiment runner may use privileged object positions to score trials. The deployed observation must contain only available images and estimated state. Save the exact messages at every boundary so a failure can be replayed.

CHAPTER 4 FLIGHT SIMULATION AND THE CLASSICAL MISSION

Build a controllable litter field

Create a small world before a photorealistic one. Begin with a 20 m × 20 m level plane, three litter categories and a few harmless distractors. Dimensions are example settings. Match the real object sizes and the intended camera resolution; miniature objects or oversized meshes produce misleading training data.

Represent the scene as data

```
layout_id: sim_train_0042
seed: 42
field_size_m: [20, 20]
objects:
  - instance_id: bottle_mesh_07
    class: bottle
    position_enu_m: [3.0, 1.0, 0.0]
    yaw_rad: 0.7
    physical_size_m: [0.07, 0.07, 0.22]
condition_id: grass_overcast_easy
```

Generate a layout file first, then generate or spawn the SDF world from it. Use the same file for scoring, but expose only images and estimated state to the policy. Store mesh and texture versions and licences. Keep the normal research software separate from privileged ground-truth access.

Increase difficulty in a curriculum

Level	What to vary	Why
0	One object, plain ground, fixed light.	Debug actions, projection and success detection.
1	Several categories, target on either side.	Require language based target selection.
2	New object meshes, textures and layouts.	Measure appearance and spatial generalisation.
3	Grass, shadows, clutter and partial occlusion.	Approach the real perception problem.
4	Measured blur, pose noise, delay and drift.	Assess transfer stresses after the easy task works.

Randomise plausibly

Start with a small set of documented conditions. After a real camera pilot, randomise around observed exposure, colour, viewpoint and blur ranges. Variation is useful only when it preserves label correctness and physical meaning. Do not independently scramble quantities that belong together, such as focal length, image crop and calibration.

Completion test

Recreate the same layout from a seed. Render a contact sheet of at least 20 scenes and check sizes, floating objects, occlusions and incorrect shadows. Count class balance and visible object pixels before generating trajectories.

CHAPTER 4 FLIGHT SIMULATION AND THE CLASSICAL MISSION

Turn the simulator into an experiment

An episode runner starts a scene, executes one trial, scores it and saves the result. This infrastructure is needed for imitation learning and evaluation; it does not imply RL.

Make these operations explicit

```
reset(layout, start_pose, instruction, seed)
observe() -> image + estimated state + timestamps
apply(action) -> accepted or rejected + reason
step_until_next_observation()
finish(reason) -> outcome + logs + metrics
```

A clean restart of the simulator and PX4 per trial is an acceptable first reset implementation. It is slower but easier to validate than teleporting an armed vehicle. Optimise reset only after you can prove it resets estimator state, object state, clocks, map memory, action queues and random seeds.

Define a single inspection episode

Start above a candidate area with at least two potential targets visible. Give an instruction such as “centre above the can”. Finish on success, timeout, leaving the allowed region, invalid state or an intervention. Save the terminal reason even if the trial is discarded from training. Evaluation must retain all assigned trials.

Example success definition: the correct target lies within 10% of the image width and height around the centre, altitude is inside its allowed band, horizontal speed is below a declared threshold, and all conditions hold for one second. These thresholds must be agreed using validation trials. Use simulator truth for scoring target identity, never as a policy input.

Write the first runner before adding models

```
for trial in manifest:
    reset(trial)
    while not terminal:
        obs = observe()
        action = classical_policy(obs, trial.instruction)
        apply(action)
        log_step()
    score_and_save(trial)
```

Completion test

Run ten trials from one manifest, including absent targets and timeouts. Re-run them and explain any differences; rendering and physics may not be bitwise deterministic. Check that no target, map entry or queued action survives reset. Report both seed and resolved scene configuration.

CHAPTER 4 FLIGHT SIMULATION AND THE CLASSICAL MISSION

Complete the classical mission first

Build the complete find, analyse and map loop with a scripted survey and a classical local inspection policy. This gives the VLA a fair comparator and exposes failures unrelated to learning.

Survey then inspect

Generate a lawnmower path inside the field polygon, with margins for turns. Estimate camera footprint from height and field of view. If the usable cross-track footprint is W and desired overlap is o , use line spacing no greater than $W \times (1 - o)$. At width 8 m and overlap 30%, spacing is at most 5.6 m. Shrink the usable footprint if edge detections are weak.

During survey, detect candidate litter, project locations and maintain tracks. After a candidate is selected, approach its estimated region, visually centre it, save a sharp view and update its map entry. Resume the unfinished survey path. Add an inspection budget to prevent one ambiguous object consuming the whole flight.

Use a transparent inspection baseline

Select the requested category from the detector’s visible candidates. Project the target centre to the ground using estimated pose and calibration. Request a bounded nearby waypoint towards that position. Re-detect and update. If the target disappears, hold and allow a brief reacquisition interval; then report failure. This is a baseline policy using the existing autopilot, not a new MPC controller.

Subsystem	First completion test
Coverage	A map of observed ground footprints with the declared image-quality conditions.
Detection	Recall versus object pixel size, with leaves and stones as negatives.
Inspection	Correct item selected and centred; failure reason saved.
Mapping	One item per track, evidence image and a location in the chosen reference frame.
Mission	Ten development layouts run end to end with complete logs and scores.

Diagnose before improving

If mission recall is low, use an oracle detector once as a diagnostic to separate coverage from perception. If map error is high, use true pose once to isolate geometry from state estimation. Keep these oracle runs labelled and out of the deployable baseline results.

Complete and test this baseline before replacing its decisions and inspection controller with the unified VLA. Both methods use the same route memory, projection, map association and flight limits. The baseline detector is a comparator component, not an extra model in the unified target system.

CHAPTER 4 FLIGHT SIMULATION AND THE CLASSICAL MISSION

Pause inspect mark and resume

Treat the flight as one continuing mission. Keep a deterministic state machine, but let accepted VLA event proposals choose the inspection transitions. A flight fault or pilot override always takes precedence.

Current state	VLA proposal	Manager action
SURVEY	CONTINUE or DETOUR	Continue route, or save progress and lock one image-grounded candidate.
INSPECT	CONTINUE or ANALYSE	Apply bounded movement, or enter a validated hold for evidence assessment.
ANALYSE	MORE_VIEW, MARK or RESUME	Acquire another view, validate a map update, or abandon with a recorded reason.
INSPECT / ANALYSE	RESUME	Close the inspection; rejoin the first unfinished survey segment.
Any state	HOLD	Hold through the existing flight system; log the reason and expired work.

Save progress before the first detour command

Snapshot route_id, route_version, segment index, along-segment progress, measured coverage bitmap, task_epoch and active track. Coverage comes from accepted camera footprints, not the nearest waypoint index. Select a conservative rejoin point before the first uncovered segment; a detour must not silently mark skipped ground as surveyed.

Use transactional decisions

Give every proposal an event_id, observation_id, map_version, task_epoch and expiry. Accept a DETOUR once, then advance the task epoch. Reject repeated events and late actions from a previous inspection. Validate its image region and project it to a reachable ground region before leaving the path.

MARK commits an evidence-linked record idempotently using event_id and track_id. RESUME remains possible after a failed inspection: store unknown or failure rather than inventing a category. A budget or flight fault may force a logged return or landing through the lab flight procedure; record this separately from a learned decision.

CHAPTER 4 FLIGHT SIMULATION AND THE CLASSICAL MISSION

Lab 11 Build the mission manager

Research build exercise. Implement the manager before connecting learned decisions. The supplied packet contains example records in templates/single_vla/; it does not contain a completed mission manager.

Build and replay the state transitions

- Create mission_manager.py with SURVEY, INSPECT, ANALYSE and HOLD states. Keep the route/coverage snapshot and append-only event log outside the model.
- Implement propose(event, observation, context) and accept_or_reject(proposal). Return the new state, execution request and reason; never send flight commands directly from model inference.
- Replay a scripted sequence: continue survey, detour, move, analyse, mark, resume. Use an explicit resume request after the map commit.

```
if proposal.task_epoch != current.task_epoch: reject("old_task")
if proposal.is_expired or observation.is_stale: hold_and_log()
if event_id in applied_events: return previous_result
if valid_transition and within_budget_and_bounds:
    commit_event_and_state_atomically()
    publish_execution_request()
```

Required fault cases

Repeat DETOUR and MARK messages; deliver a delayed action after RESUME; remove the camera stream; select an already mapped object; lose the target; interrupt between map commit and resume. Include two items of the same class and a partial survey segment.

Pass when: a duplicate mark creates no second object, a detour preserves unfinished coverage, late proposals cannot take control, and resumption never skips unobserved ground. Save before/after state records and a coverage plot. Review these tests in simulation with the supervisor before integrating a learned event head.

The manager validates and executes the VLA's choices. It must not silently replace them with a rule-based inspection selector in a run reported as the unified model. Log every override and fallback.

CHAPTER 4 FLIGHT SIMULATION AND THE CLASSICAL MISSION

Detect and analyse within the same VLA

The target system uses the same visual backbone for candidate grounding, mission decisions, local actions and semantic labels. The classical comparator may have a separate detector. Keep these architectures clearly labelled.

Make a distant candidate an observable prediction

Begin with one selected image region per decision: `litter_present`, `bbox_xyxy` in normalised image coordinates, `category` including `unknown`, and `observation_id`. A shared visual-feature grounding head or supported token output must learn these fields from annotated images. Do not give the model the true ground location or preselect only the correct crop.

Measure candidate recall against object pixel size and viewing angle. A downward camera sees only its footprint; a “far” target must already be observable. If resolution is inadequate, test a permitted altitude change, larger model input or a fixed tiling scheme using the same encoder. These change compute and training, so document and benchmark them.

Separate suspicion from a confirmed label

A tiny bright patch can justify an approach without justifying “plastic bottle”. Use `unknown` until sufficient evidence is available. After approach, the same model evaluates the better image and either proposes `MARK` or `MORE_VIEW`. Limit attempts and record abandoned candidates.

```
{
  "event": "MARK",
  "track_id": "track_17",
  "category": "can",
  "attributes": ["silver", "crushed"],
  "evidence_id": "frame_0921",
  "unknown_fields": [],
  "bbox_xyxy": [0.42, 0.31, 0.55, 0.48]
}
```

The mapper obtains metric position from this region, capture-time pose and calibration; the VLA does not invent GPS coordinates. Category and attribute heads use a fixed label vocabulary initially. Add free-form descriptions only if their correctness can be evaluated.

Evaluate before flying

Use held-out real and simulated images containing small litter, leaves, stones, reflections and ambiguous items. Report recall by size, false candidates, category macro-F1, unknown rate and extra-view decisions. Calibrate confidence on development data. Count resize, crop and tiling failures as part of the deployed pipeline.

CHAPTER 4 FLIGHT SIMULATION AND THE CLASSICAL MISSION

Turn observations into a litter map

For a level field and a calibrated nadir camera, projection is a manageable geometry problem. It does not require a new SLAM system if the drone already provides usable pose estimates. Start with that assumption and measure its error.

Check whether objects are resolvable

$$GSD \approx H / f ; \quad w \approx L \times f / H$$

Here H is camera height above the plane, f is focal length in pixels, L is object width in metres and w is its projected width in pixels. At $H = 10$ m and $f = 1,000$, a 0.10 m object spans about 10 pixels. Reducing image width by four reduces that to about 2.5 pixels. This explains why full-resolution detection and smaller VLA inputs may need different image paths.

Project a detection

Undistort the image point; form its camera ray using the inverse intrinsic matrix; rotate and translate that ray through the calibrated camera-to-body and body-to-world transforms; intersect it with the ground plane. Use pose at image capture time. Reject nearly horizontal rays or invalid intersections. The camera centre and object contact point are not always the bounding-box centre.

Validate and represent uncertainty

Project surveyed reference points and compare with their independent positions. Test true simulator pose and noisy estimated pose separately. Perturb calibration, pixel location, ground height and pose within measured uncertainty to estimate a location uncertainty region. Do not average away a systematic GNSS bias. A category confidence is different from spatial uncertainty.

The toy map uses a category key because it has one item per category. Replace this with instance tracks before evaluating fields containing multiple cans or bottles.

Associate repeated sightings

Maintain object tracks using position uncertainty, appearance, time and category evidence. Closely spaced items must remain separable: a fixed 1 m merge radius can collapse distinct objects. Keep links to every supporting image and log merges so they can be audited. Compare naive clustering with your chosen rule on a dense-item test.

Export and score

Use one GeoJSON Feature per item within a FeatureCollection [27]. Convert local positions to the declared geographic reference; write longitude before latitude. Store object ID, class, evidence, first/last seen times, uncertainty and calibration version in properties. Keep the local metric coordinates in the experiment log for distance scoring.

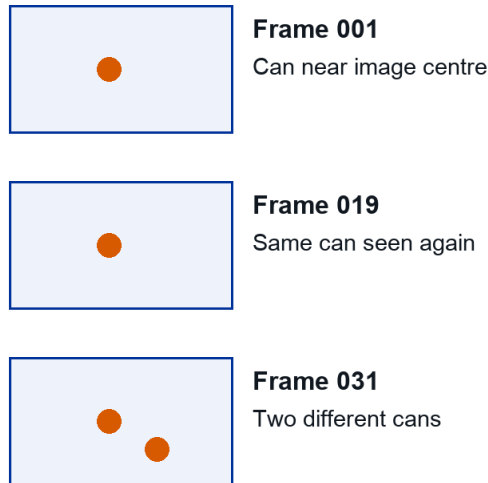
A small map error among a few matched detections can hide poor recall. Always report map recall, false positives and duplicate rate alongside median and 95th percentile position error.

CHAPTER 4 FLIGHT SIMULATION AND THE CLASSICAL MISSION

Keep object identity across repeated views

Several detections can refer to one physical item. Conversely, two nearby cans are still two items. A useful map keeps persistent instance tracks and retains the supporting images.

MULTIPLE VIEWS



ONE TRACK PER PHYSICAL ITEM

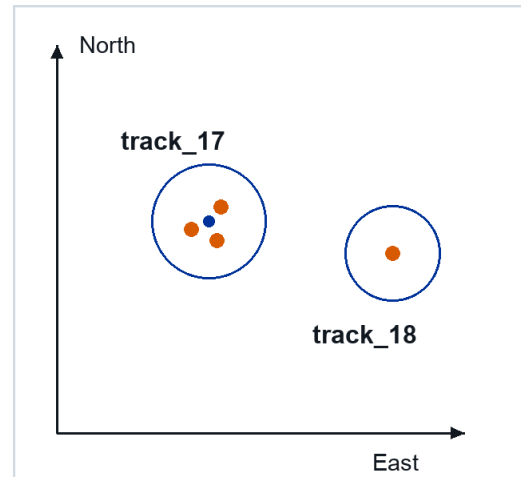


Image point + capture-time pose + calibration

Project to the ground, then associate observations using position uncertainty and appearance.

Illustrative uncertainty regions; same category does not mean the same object.

Figure 8. Schematic map association. Orange dots are projected sightings and blue circles illustrate uncertainty regions, not measured confidence intervals.

Read the association example

Three nearby sightings support track_17. A different can belongs to track_18. Geometry, uncertainty, appearance and time inform association. A category-only key would incorrectly merge these two cans.

Validate before trusting the map

Use a surveyed scene with repeated views and two closely spaced objects of the same class. Count correct tracks, duplicates and incorrect merges. Score location error only alongside recall and precision.

CHAPTER 5 DATASETS AND VLA TRAINING

Connect images skills and mission decisions

One shared model needs several kinds of supervision. Store related views as linked records, with mission and session IDs that prevent train/test leakage. Public photos support visual learning but do not provide the decisions and movements of this drone.

Data family	Labels	Use
Perception and analysis	Regions, presence, category, attributes, visibility and unknown.	Train shared grounding and semantic outputs.
Inspection skills	Images, instruction, state and expert movement.	Train bounded local actions.
Complete missions	Context, chosen event, selected region, movement masks, outcomes and resume record.	Learn when to detour, inspect, mark and resume.
Independent evaluation	Surveyed layout and classes; complete frozen-policy logs.	Score map quality and coverage under budget.

Public data and the included teaching data

Use UAVVaste [1] and TACO [19] for relevant imagery and class definitions after checking licences. The supplied 210 toy episodes teach local centring only. They contain no survey interruption or resumption labels and cannot by themselves train the final model.

Plan collection in stages

Begin with 50–100 clean simulated inspection episodes and 200–500 reviewed real frames to check interfaces and visibility. Then collect complete simulated missions with negative candidates and multiple inspections. A development pilot of 20–30 missions can reveal schema problems; choose later data counts from learning curves rather than assuming sufficiency.

If field access permits, collect 30–100 supervised real inspections and complete survey sessions across different days and layouts. Reserve real test groups before adaptation. These are planning budgets, not guarantees of statistical power or successful training.

Write a label handbook

Define bottle, can, paper, non-litter and unknown; specify visibility, valid image regions, physical instance IDs, candidate-worth-inspecting decisions and abandonment reasons. Review ambiguous events with the supervisor. Multiple reasonable decisions may be valid; record that ambiguity instead of treating one expert choice as universal truth.

CHAPTER 5 DATASETS AND VLA TRAINING

Generate useful simulated demonstrations

An expert demonstration is a time-aligned sequence showing what command should follow each observation. The first expert can be scripted and use simulator truth. The student policy must receive only quantities that will exist on the drone.

Build the expert in small steps

1. Sample a layout, target instruction and starting pose with the target visible.
2. Use the requested object's true location to define the expert goal.
3. Compute a bounded next position towards that goal using the declared action contract.
4. Let PX4 move the simulated drone; record fresh images and estimated state.
5. Recompute at the policy sampling period until centred, timed out or failed.

For position increments, label the exact increment relative to the recorded observation anchor. Store the proposed expert action, the accepted setpoint after limits, and any rejection. Train on the accepted deployable action for ordinary demonstrations. Keep rejected proposals in diagnostics. Do not train on measured velocity when the intended label is a commanded position.

Teach selection and recovery

Balance target categories and target positions. Show a bottle and a can together, then vary which one the instruction requests. Include starting offsets, modest drift, temporary loss of visibility, no-target cases and completion holds. Use clean recoveries as demonstrations; keep irrecoverable or unsafe expert failures outside the imitation targets and retain their metadata.

An oracle can select a hidden target that an RGB policy cannot infer. Exclude such starts from the initial visible-target skill, or label a visible search/reacquisition behaviour. Do not ask the policy to learn information unavailable in its observation.

Audit before generating thousands of trials

Check	Evidence
Action meaning	Overlay arrow or waypoint; signs and units agree with motion.
Time alignment	Show frame capture, expert decision and command application on a timeline.
No privileged leakage	Policy loader excludes true target coordinates, object IDs and future frames.
Diversity	Counts by category, target position, mesh, scene and instruction.
Termination	Success, absence, timeout and failure counts; no truncated silent episodes.

Review at least ten complete episodes manually, including failures. Save the audit and generator version as part of dataset release 0.1. An error discovered after generation requires a new version, not an undocumented patch.

CHAPTER 5 DATASETS AND VLA TRAINING

Label complete mission decisions

A mission episode begins at the declared survey start and ends at completion, budget expiry or intervention. Inspection segments are children of that episode. Split by mission, layout, assets and real session before sampling frames.

What the expert must demonstrate

Collect ordinary survey frames, early candidate sightings, decisions to ignore distractors, justified detours, unsuccessful approaches, repeated views, map commits and resumption. Vary distance, object size, image quality, remaining budget and whether a similar item has already been mapped.

```
mission_id, segment_id, decision_id, observation_id
mode_before, route_version, coverage_summary, track_summary
remaining_budget, visibility, expert_event, selected_bbox
expert_action, action_valid, category, category_valid
evidence_ids, mode_after, resume_segment, terminal_reason
```

Avoid action and information leakage

During SURVEY, route-following setpoints are produced by ordinary navigation code; set `action_valid=false` for the learned movement loss. Train DETOUR decisions from what was visible at capture time. During INSPECT, label expert movements and enable the action loss. Enable category/attribute losses only where the evidence and annotation support them.

Do not train a distant-frame category using a close-up observation that occurs later unless it is explicitly a weak label experiment. Likewise, future success and surveyed coordinates belong to evaluator records. A simulated expert may use privileged truth for supervision, but its labelled decision must be feasible from the supplied observation; flag unobservable cases.

Sample rare events without corrupting evaluation

Survey continuation frames will dominate raw recordings. Use balanced training batches or event weights, and report the sampling scheme. Keep natural event frequencies in validation and test. Add paired contexts where the same image merits a detour with ample budget but a resume/ignore decision when the item is already mapped or the budget is exhausted.

Deliverable: a mission dataset card, label handbook, grouped split manifest and ten replayed event sequences. Use `templates/single_vla/` as examples; replace their illustrative values with recorded evidence.

CHAPTER 5 DATASETS AND VLA TRAINING

Collect real data in three passes

Use clean placed objects and a trained pilot under the lab’s field procedure. Plan data collection around learning and evaluation questions; collecting a long unstructured video is less useful than controlled variation and complete metadata.

Pass A Measure the camera and field

Calibrate intrinsics at the actual resolution and focus setting [28]. Measure the camera-to-body transform. Record ground height, the local origin and a set of surveyed reference points with their measurement uncertainty. Photograph every physical item and assign a persistent instance ID. Record camera settings, weather, lighting, ground type and time.

Pass B Collect perception and mapping data

Fly repeatable survey lines at several approved heights and speeds. Capture sharp hover views as well as motion views. Include empty ground, distractors and tightly spaced litter. Log raw images, estimated pose, capture times and flight telemetry. Independently survey the items; do not use the drone’s predicted map as its own ground truth.

Pass C Collect actions and mission events

Have a validated classical inspection policy or an authorised operator issue the same position or velocity command format the VLA will use. Pair each observation with that command. Manual stick inputs are not interchangeable with local waypoints: if stick-only flights are used, they require a documented conversion or relabelling procedure and introduce additional uncertainty.

Store operator interventions, rejected actions and the original task instruction. Include the mission context and reviewed DETOUR, ANALYSE, MARK and RESUME events; movement logs alone cannot train the event head. When using a scripted expert, inspect its real behaviour before accepting its trajectories as good supervision. Record target-not-visible and failed episodes separately rather than making every recorded flight a success.

Split by collection session before tuning

Real subset	Use
Development training	Fit camera/domain adaptation and fine-tune models.
Development validation	Choose thresholds, prompts, checkpoints and simulator randomisation ranges.
Locked test	New sessions, layouts and held-out item instances; evaluate the frozen system once the protocol is fixed.

Keep field dates and physical objects separated where the research question requires it. If only one site is available, report within-site generalisation. More frames from the same flight do not provide more independent field trials.

After each session

Count and recover all placed objects, back up logs, check missing timestamps and calibration IDs, and review a contact sheet before the next flight. Keep an annotation queue with hard examples and a manifest connecting each image, physical item, session and dataset split.

CHAPTER 5 DATASETS AND VLA TRAINING

Store align and split the data

A minimum episode record

```
episode_id, domain, session_id, layout_id, split
object_instance_ids, instruction_id, instruction_text
image_path, capture_time, receive_time, calibration_id
estimated_position_ned_m, velocity_ned_m_s, attitude
action_schema, anchor_pose, expert_action, accepted_action
mission_id, segment_id, mode, context_version, expert_event
event_id, selected_bbox, evidence_ids, head_validity_masks
action_send_time, intervention, terminal_reason
software_commit, generator_version, sensor_validity
```

Store ground-truth positions and expert-only variables in a separate evaluator file. The training loader reads an allowlist of deployable features. Use checksums and a versioned manifest so a checkpoint always points to the exact data and calibration it used.

Align a frame with the decision it informed

If a frame was captured at $t = 10.00$ s, received at 10.04 s and used for a command sent at 10.10 s, retain all three times. The target action is the expert decision for that observation, not an arbitrary command from the nearest later frame. If you resample to 2 Hz, define the sampling rule and represent holding periods explicitly.

Mask padded frames at episode ends and reset model history between episodes. Never form action chunks across episode or mission-mode boundaries. Mark which heads have valid labels at each step; route-following samples do not train the inspection action head. If using data-derived normalisation, compute it on the training split only and save it with the checkpoint. The supplied GPU exercise uses declared fixed physical scales instead; record that choice explicitly.

Create grouped splits

A suggested starting allocation is 70% training, 15% validation and 15% test, assigned by groups rather than by frame. Use separate held-out tests for new layouts, new assets or physical items, new instruction wording, and a combined shift. A seed alone does not guarantee independence if the same scene or item appears elsewhere.

Automated data checks

Check monotonic times, image existence, finite state, frame conventions, action bounds, duplicate hashes, consistent dimensions, calibration versions and split overlap. Replay a random sample after every export. Count missing data explicitly and preserve the exclusion reasons.

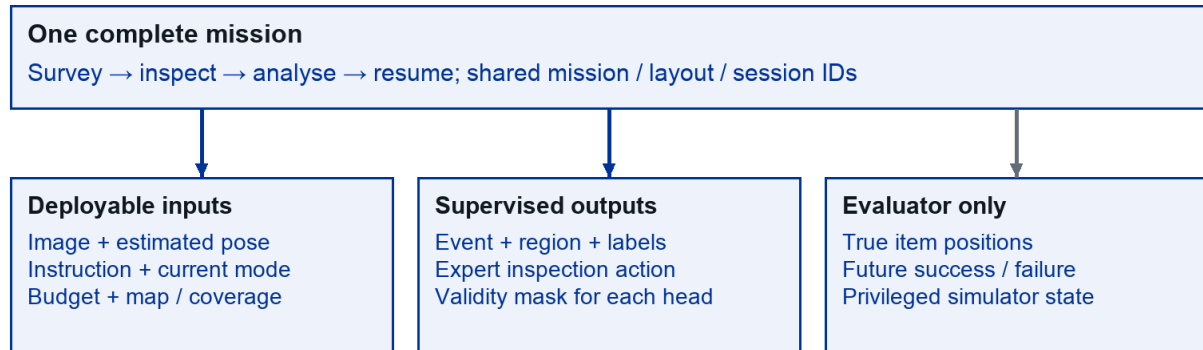
Estimate storage before collection

One hundred 30-second episodes at 10 images/s contain 30,000 images. At an observed average of 200 kB per compressed frame, that is about 6 GB, before logs and backups. Measure actual file sizes and compute your budget. Retain full-resolution evidence separately from model tensors; downsampling cannot recover tiny litter later.

CHAPTER 5 DATASETS AND VLA TRAINING

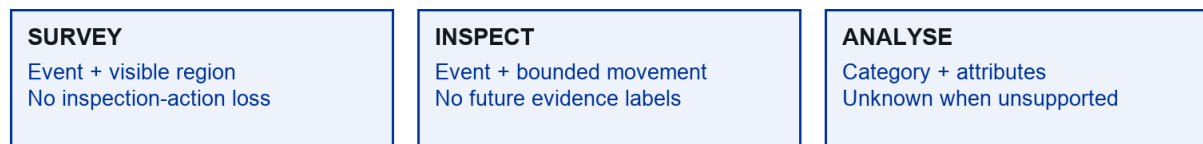
Separate mission labels from evaluator truth

A complete mission contains survey, inspection, analysis and resumption segments. A record carries inputs, supervised outputs and masks indicating which output heads have valid labels.



Never a deployed model input

VALID LABELS DEPEND ON THE MISSION MODE



Split whole missions and field sessions before sampling frames.

The bundled 150 / 30 / 30 episodes are local teaching skills, not complete missions.

Figure 9. Proposed complete-mission dataset. Ground truth stays outside deployed inputs. The bundled 150/30/30 split contains local teaching skills only.

Apply the right loss to each record

SURVEY frames supervise candidate regions and events, while ordinary route commands are excluded from the learned inspection-action loss. INSPECT frames can supervise movement. Semantic labels require sufficient visible evidence. Split whole missions and field sessions before sampling.

Keep the two episode types distinct

Use the supplied 210 toy episodes to learn the movement workflow. Collect separate complete mission episodes for interruption and resumption. An increase in toy action accuracy is not evidence of mission-level reasoning.

CHAPTER 5 DATASETS AND VLA TRAINING

Train the first inspection VLA

This page builds the movement pilot first. Start with SmolVLA as a compact candidate [6, 23, 39]. Its public checkpoint comes from manipulation tasks. It does not already understand drone action units or embody a flight controller. Adapting its state and action interfaces is required. The event, grounding and analysis heads for the final mission model are introduced on the following page.

Stage 1 Freeze the data contract

Use the full local RGB view, the instruction, estimated horizontal velocity, height and orientation. If yaw is variable, encode it without an angle discontinuity, for example sine and cosine. Avoid global target coordinates as inputs. Predict the two position increments from page 19. Use a single-step horizon first; keep fixed altitude and yaw outside the learned output.

Stage 2 Adapt the model deliberately

Map your image and state fields to the selected implementation. Adapt or reinitialise state/action projections for the new dimensions and units; mask unused padded dimensions if the implementation uses them. Preserve compatible pretrained visual and language weights. Record which weights are loaded, replaced, frozen and trained. Verify the forward output shape before launching training.

Stage 3 Overfit a tiny clean subset

Use 10–20 uncomplicated demonstrations with several target directions. Disable unnecessary augmentation. Check that training loss decreases, denormalised outputs have correct units and the policy can replay the easy behaviour in closed loop. If it cannot, fix alignment, masks, action scaling or model wiring before collecting more data.

Stage 4 Train and select on development data

Start by training the action module and new projections with the visual backbone frozen, if supported by the chosen implementation. Use the model's supported fine-tuning recipe and optimiser defaults as the first configuration. Adjust batch size to measured workstation memory, using gradient accumulation if needed. A 24 GB GPU is a planning resource, not a guarantee that every configuration fits.

Evaluate checkpoints on fixed validation episodes. For the research model, declare a validation criterion combining correct-target success and intervention rate, with action loss as a diagnostic. The supplied GPU pilot instead saves the best validation flow-loss checkpoint; examine its closed-loop validation before selecting the research configuration. If perception adaptation is clearly needed, unfreeze a limited portion or use a supported adapter method as a separate experiment. Record training time, peak memory, data count and seeds.

Stage 5 Package for deployment

Save weights, normalisation, preprocessing, action schema, calibration assumptions and code version together. Run the same stored observation through workstation and Orin inference and compare outputs within a declared numerical tolerance. Never omit the normaliser because the model weights loaded successfully.

CHAPTER 5 DATASETS AND VLA TRAINING

Train one model with several supervised outputs

The target is one checkpoint with one shared image/language backbone. Add explicit mission-context projections and event, grounding and semantic heads alongside the action expert. Native SmolVLA in the supplied scripts does not implement these additions [6, 42].

Build the outputs in a controlled order

- First reproduce the movement-only pilot in Labs 9 and 10. Keep that checkpoint as a reference.
- Add mode, remaining budget, coverage and track-summary inputs with fixed units and validity masks. Document truncation when the map contains more tracks than the input budget.
- Add an event classifier and one-region grounding head. Initially predict category and visible attributes with fixed vocabularies, including unknown.
- Verify each head on a tiny labelled subset, then train the heads together on mixed image, skill and mission batches. Keep the pretrained backbone frozen first; unfreeze a limited portion only if validation supports it.

Use an explicit masked objective

```
L = w_event * CE(event)
  + w_region * region_loss(valid_regions)
  + w_semantic * semantic_loss(valid_labels)
  + w_action * flow_matching_loss(valid_inspection_actions)
```

Normalise each loss by its number of valid labels in the batch. Use zero contribution when that head has none. Tune weights on validation data and save them. Classification and region outputs are not flow-matching movements. Keep the native continuous-action objective where supported; do not relabel the entire training process as RL.

Prevent forgetting and inconsistent proposals

Retain skill batches while adding mission decisions, and measure movement regression. Define which outputs are valid for each mode. A MARK prediction cannot simultaneously execute a new move; the manager applies one consistent event and rejects incompatible fields. Keep explicit image and context versions across asynchronous inference.

Select the checkpoint using a declared validation mission score subject to command-validity limits, while reporting per-head losses and skill success. Compare at least three training seeds if feasible. Training cost, model size and first/last-layer choices are part of the result, not assumed properties of the name SmolVLA.

CHAPTER 5 DATASETS AND VLA TRAINING

Lab 9 Adapt SmolVLA to the action contract

Movement pilot only: this lab does not implement mission, grounding or semantic heads. Add those in Lab 12.

SmolVLA is the first pretrained VLA candidate because it is compact enough to make an embedded evaluation plausible. It was not trained as a ready-to-use litter-search drone controller. This thesis must test the transfer. The GPU scripts in integration/ are source-reviewed but have not been executed in this package; this lab is a supervised integration starting point. [6, 39]

Install a separate environment

```
python3 -m venv .venv-vla
source .venv-vla/bin/activate
python -m pip install \
    "lerobot[smolvla] @ git+https://github.com/huggingface/lerobot.git@v0.4.4"
python -m pip install -r requirements.txt
```

The lab must first provide a CUDA-compatible PyTorch installation and verify `torch.cuda.is_available()`. Run the commands from the kit root. Initial model loading may download weights and a tokenizer. Save the resolved package and model revisions with the run. [42]

What the supplied adaptation changes

It loads the pretrained model, describes one full RGB camera and seven state entries, and predicts two horizontal position increments. It resets the state and action input/output projection layers, retains the pretrained representation and action expert, uses one action per decision and creates fresh processors.

State order is [north velocity, east velocity, height, roll, pitch, sine yaw, cosine yaw]. Divide these by [1, 1, 5, 1, 1, 1, 1]. Divide action labels in metres by 0.5; multiply outputs by 0.5 before the command checker. These explicit physical scales avoid reusing robot-arm normalisation statistics.

The model retains its internal padded dimensions. Input image shape is $B \times 3 \times 96 \times 96$, state is $B \times 7$ and action label is $B \times 1 \times 2$. The supplied training loop uses SmolVLA's native flow-matching loss, not the tiny model's MSE objective.

Pass when: the supervisor can show a finite training step, nonzero trainable gradients and two finite action outputs after saving and reloading the checkpoint. This is the first GPU smoke test, not a navigation success claim.

CHAPTER 5 DATASETS AND VLA TRAINING

Lab 10 Train and evaluate the VLA

Movement pilot only: the supplied command trains the continuous action expert. Build the unified mission outputs in Lab 12 after this pilot works.

```
python -m integration.train_smolvla \
  --data runs/data --out runs/vla-smoke \
  --steps 100 --batch-size 2
```

Expected structure: history.json, a best checkpoint with processors, and drone_interface.json. No expected numeric loss is claimed because this GPU exercise was not run here. Inspect finite values, action shape and physical scaling before allocating a longer job.

```
python -m integration.train_smolvla \
  --data runs/data --out runs/vla-dev \
  --steps 2000 --batch-size 2
python -m integration.evaluate_smolvla \
  --checkpoint runs/vla-dev/best --split val \
  --out runs/vla-val
```

The script saves the best validation flow-loss checkpoint. Examine validation closed-loop success as well. When the setup is frozen, evaluate the test split once and run the language interventions. The next page gives the final evaluation commands and optional LeRobot dataset exporter.

Know what this test still omits

The toy world waits while inference runs. A model taking two seconds can therefore appear to control a nominal 2 Hz simulation. That result is not evidence that the model meets a 0.5 s deadline. The evaluator records inference latency separately; later test the asynchronous flight pipeline under realistic delay.

A sensible failure response

For a shape error, print image, state and action tensor shapes before the policy call. For missing normalisation statistics, verify that the new processors are loaded. For memory failure, try batch size one. For poor behaviour, inspect units, frame signs, image content and task coverage before increasing model size.

The 96-pixel teaching image is intentionally crude. Resizing it inside SmolVLA cannot recreate a tiny litter item that was never visible. In the research simulator, select resolution and altitude from measured object pixel size and detection performance.

Deliverable: a reloadable checkpoint, full environment record, validation and test protocol, normal and changed-language results, and a list of differences still separating the toy task from the flight simulator.

CHAPTER 5 DATASETS AND VLA TRAINING

Evaluate and export the adapted checkpoint

These commands evaluate the local movement pilot. They do not measure survey decisions, mapping or the completed single VLA mission.

Run the first GPU evaluation on validation scenes. Confirm finite actions, correct physical scaling and target-directed behaviour before opening the final test. These scripts use full images and instruction strings, unlike the colour-feature teaching model.

```
python -m integration.evaluate_smolvla \
  --checkpoint runs/vla-dev/best --split val \
  --out runs/vla-val
```

Freeze then evaluate

Record the checkpoint, task conditions and test manifest. The following normal and changed-language trials use the same held-out starts. In the changed-language condition, the evaluator retains the original goal while the policy receives the next category; report that intervention explicitly.

```
python -m integration.evaluate_smolvla \
  --checkpoint runs/vla-dev/best --split test \
  --out runs/vla-test
python -m integration.evaluate_smolvla \
  --checkpoint runs/vla-dev/best --split test \
  --language swapped --out runs/vla-swap
python -m integration.evaluate_smolvla \
  --checkpoint runs/vla-dev/best --split test \
  --language empty --out runs/vla-empty
```

Optional LeRobot interchange dataset

```
python -m integration.export_lerobot \
  --data runs/data --split train --out runs/lerobot-train
```

Export validation and test to distinct destinations if needed. The exporter writes local data and does not upload it. The supplied training loop reads the raw observation/action records directly; conversion is therefore optional, not an unexplained prerequisite.

Preserve the deployment interface

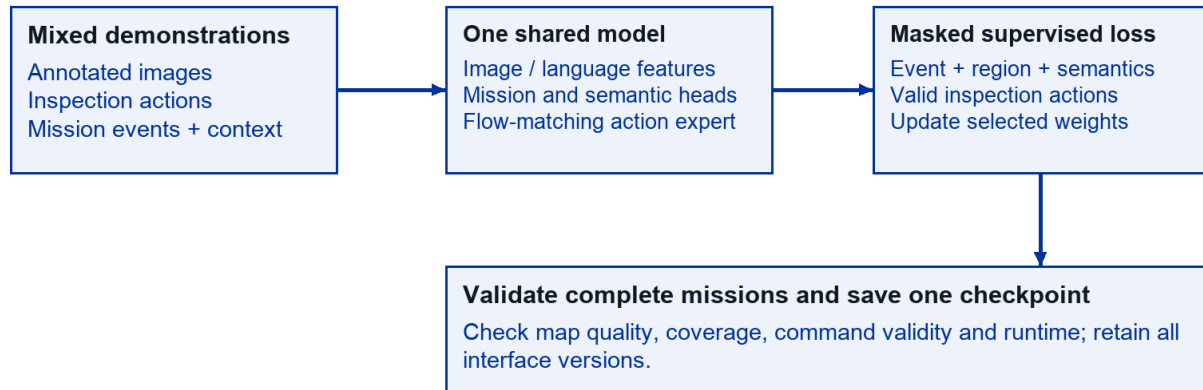
Keep the checkpoint, fresh processors and `drone_interface.json` together. Its fixed physical scaling is part of the model. The toy evaluator pauses simulated motion during inference, so report latency separately and then repeat with real-time pacing on the flight simulator and Orin. No GPU success rate is claimed by this guide. [42]

CHAPTER 5 DATASETS AND VLA TRAINING

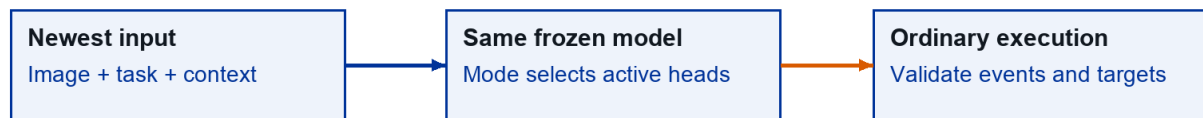
Train and deploy one shared model

Different labels supervise different outputs of the same backbone. The final model must demonstrate the combined behaviour in closed-loop missions, beyond the movement-only teaching pilot.

WORKSTATION TRAINING



ORIN INFERENCE



Research design: the supplied trainer currently covers the movement pilot only.

Figure 10. Proposed unified supervised training. Mission heads are research additions. The included SmolVLA trainer implements the movement pilot only.

During training

Mix annotated images, inspection demonstrations and complete mission decisions. Apply validity masks and retain movement batches when adding event learning. Select on development mission results and save the data contract, head definitions and scaling with the checkpoint.

During deployment

Load one checkpoint on the Orin. Enable outputs according to the validated mission mode. The independent manager, geometry and flight supervisor execute accepted proposals. No online weight updates are required.

CHAPTER 5 DATASETS AND VLA TRAINING

Evaluate what the VLA actually learned

A minimal experiment sequence

Run	Change	Question
Sanity	Tiny clean data; easy starts.	Does the implementation learn the intended action units?
Main	Full training split; fixed architecture.	Does the policy generalise to held-out trials?
Language control	Same scene, different requested item.	Does the instruction change the selected target?
No language	Empty or shuffled instructions.	Could image-only shortcuts explain performance?
Transfer	Frozen policy on measured real conditions.	Where does performance deteriorate?

Make language necessary

Put two plausible targets in the same view and balance which is requested. Swap “bottle” and “can” while keeping the scene and start pose fixed. Use held-out paraphrases after the vocabulary is understood. Include absent targets and score whether the system correctly declines or times out. Do not crop only the correct object before a target-selection experiment.

Then test recovery

Evaluate modest unseen offsets, partial occlusion, motion blur and delay separately before combining them. Save image/action traces for each failure. If the policy repeatedly leaves expert states, use simulation-only corrective collection: run the learner, query the expert at visited states, add the labelled examples and retrain [33]. Keep the frozen test set out of this loop.

Action chunks are an optimisation to test later

An action chunk predicts several future setpoints at once. It can reduce inference overhead but commits to motion based on older evidence. If introduced, define all future position targets relative to the original observation anchor, keep their timestamps, and execute only a validated prefix before replanning. Discard superseded chunks and any chunk from an old task.

For the first result, use one proposed action per update. When comparing horizons, keep the dataset sampling interval and action units fixed. Report chunk length, executed prefix and effective observation-to-action age. A 20 Hz executor resending setpoints is not a 20 Hz learned policy.

A training run is complete when

Its dataset version, code commit, configuration, seed, learning curves and selected checkpoint are saved; its validation and test roles are distinct; and its failures can be reproduced. Use at least three training seeds for the final main model if compute permits, and report the actual number if it does not.

CHAPTER 5 DATASETS AND VLA TRAINING

Lab 12 Integrate the unified mission VLA

Research build exercise. Extend the movement pilot; the supplied training command does not implement this lab. Use the mission dataset contract, proposed heads and manager tests to keep the scope concrete.

Implementation sequence

- Create `mission_dataset.py` with an input allowlist and per-head validity masks. Verify that hidden truth and future evidence never enter a model batch.
- Create `unified_policy.py` around one shared backbone. Return event logits, one candidate region, semantic outputs and a bounded action prediction when its head is enabled.
- Create `train_unified.py` using the masked objective. First overfit 10–20 reviewed mission snippets, including negative candidates and resume events; then train on grouped missions.
- Create `evaluate_mission.py` around the Lab 11 manager, simulator clock and complete mission log. Keep model proposals separate from accepted events and actual coverage.

Three checks before scaling up

Decision check: same image, changed mission budget or mapped-object history; the event should change where the expert policy requires it.

Grounding check: two plausible objects; changing the language priority must select the corresponding visible item.

Execution check: a detour followed by MARK and RESUME leaves one evidence-linked map entry and continues unfinished coverage.

Evidence required to pass

Run unseen complete missions with negative objects, failed inspections and repeated sightings. Compare with the tuned classical system and a version using the same learned heads with fixed decision rules. Save per-event confusion, map recall/precision, resumption errors, all overrides and camera-to-command timing.

A successful model prediction is not a successful mission. A checkpoint passes this lab only after closed-loop simulated execution and correct resumption are demonstrated. Orin and real flight remain separate later milestones. If hardware access is delayed, report that scope honestly rather than substituting offline action accuracy.

CHAPTER 6 REASONING AND ONBOARD COMPUTE

Reason through observable mission choices

Reasoning in this thesis means using visible evidence and mission context to choose a useful next action. The single VLA makes the decision; no separate VLM planner is deployed. A long explanation is neither required nor evidence that the decision is good.

Use a small event vocabulary

Predict CONTINUE, DETOUR, ANALYSE, MORE_VIEW, MARK, RESUME or HOLD, with only the fields valid for the current mode. CONTINUE follows the existing route in SURVEY and requests a bounded movement in INSPECT. The manager checks mode, epoch, age, bounds and budget before execution.

```
{"event_id": "m04_e18", "event": "DETOUR",
  "observation_id": "frame_0812", "task_epoch": 3,
  "bbox_xyxy": [0.52, 0.20, 0.60, 0.31],
  "reason_code": "need_closer_view",
  "context_version": 42, "valid_for_s": 0.5}
```

Make mission context necessary

Pair otherwise comparable observations with different remaining budgets, coverage histories and mapped objects. Include a clear new item, a known item, an ambiguous item and a distractor. The teacher policy should document acceptable decisions and their constraints, including ambiguous cases.

Evaluate decisions by what they cause

Replay is useful for event validity and grounding. Closed-loop missions are required to measure the new evidence produced by an approach, the time lost to a false detour and coverage retained after resumption. Count analysis holds and inference waits in the mission clock.

Keep short reason codes for auditing, such as need_closer_view, already_mapped and insufficient_budget. Do not score language fluency as a proxy for reasoning. An explanation can be plausible while the selected object or action is wrong.

Keep attribution clear

Compare the full unified policy with fixed event rules using its same perception, semantic and movement outputs. Remove mission context in a separate ablation. These isolate learned decisions and memory from changes in visual recognition. A second-model comparison is optional related work, not required by the target architecture.

CHAPTER 6 REASONING AND ONBOARD COMPUTE

Run one shared model on the Orin

Deploy one unified VLA checkpoint on the Orin NX 16 GB. Its shared visual/language features support mission decisions, candidate grounding, semantic labels and local movement. Train on the workstation. The autopilot runs on its existing flight controller.

Component	Starting target to measure	Placement
Mission manager and supervisor	20 Hz checks; independent of inference.	CPU process.
Unified VLA during survey	About 2 Hz monitoring; task/context input.	One GPU model instance.
Unified VLA during inspection	About 2 Hz bounded movement proposals.	Same model and action expert.
Analysis and map update	At a validated hold; bounded inference.	Same semantic heads; CPU projection/association.
Autopilot	Existing configured flight loops.	Dedicated controller.

These frequencies are proposed starting targets, not measured performance. The continuous survey executor can follow its route between model calls. During inspection, accepted setpoints may be resent by the executor while their validity holds; this is not a higher learned-policy rate.

Bound sensing and movement together

During survey, measure visible-ground distance per processed observation: approximately speed multiplied by the largest observation gap. If that exceeds useful camera overlap, small candidates can pass unseen. Set survey speed from camera footprint, processing latency and required coverage, then test missed detections under delay.

Use mode dependent work within one model

Run grounding/event outputs during SURVEY; use the action expert in INSPECT; run semantic outputs during ANALYSE. These are scheduled outputs of one checkpoint, not independent models. Any tile count, context size or iterative action sampling must have a fixed limit.

Start with a supported reference runtime and measured precision. Export or quantise only after checking output parity and mission behaviour. A compact parameter count does not guarantee adequate latency or total memory on the board. Preserve the validated hold response if an inference call stalls. [35, 41]

CHAPTER 6 REASONING AND ONBOARD COMPUTE

Budget memory for one model and the full system

The Orin shares memory between CPU and GPU workloads. Count weights, activation peaks, image tensors, caches, runtime workspaces, logging and the operating system. A model with 450 million parameters uses about 0.9 GB for FP16 weights alone; the adapted model and complete runtime will use more.

Allocation	Illustrative ceiling	What to measure
OS and ordinary services	4 GB	Camera, map, logger, manager and runtime overhead.
Unified VLA inference	9 GB	All heads, image/mission tokens, buffers and peak activations.
Reserved headroom	3 GB	Variation, fragmentation and temporary copies.

This allocation is a planning device, not a memory measurement or a claim that the model needs 9 GB. Use actual available memory and peak workload observations. Do not keep a second reasoning model or detector resident in a run labelled one VLA.

Keep queues short and versions explicit

Use a latest-frame queue and one inference owner. Attach observation and task versions to every output. Cache features only while they match the same image and relevant context; never reuse a stale visual representation across motion merely to improve the reported speed.

Run inference serially initially: survey monitoring, inspection movement, then held analysis. The CPU supervisor and flight execution continue independently. Reject late decisions rather than emptying an accumulated inference backlog. Include all held time in the mission cost.

Optimise after measuring the failure

First remove excessive copies and input sizes. Then test supported half precision, smaller context and bounded sampling steps, one change at a time. Quantisation may change events and coordinates, so repeat grounding, decision and closed-loop tests, not just tensor comparison.

Power and heat

Use power modes supported by the module, carrier and JetPack version [35, 41]. Choose a sustainable configuration based on cooling and flight energy. Record temperature and deadline failures after warm-up. A short desktop-style throughput result does not establish sustained airborne operation.

CHAPTER 6 REASONING AND ONBOARD COMPUTE

Measure the unified workload on the aircraft

The following are starting gates to test, not hardware results. Record safe motion speed, useful camera footprint and end-to-end observation age before selecting a deployed policy rate.

Measure	Initial gate	Record
Inspection observation age	p95 below 0.5 s at proposed 2 Hz.	Capture to accepted command, including all queues.
Survey observation gaps	No unobserved ground strips at chosen speed.	Maximum gap, footprint and candidate misses.
Peak shared memory	Below 13 GB with about 3 GB headroom.	Total RAM; avoid double counting GPU allocations.
Sustained operation	30 minutes without crash or persistent deadline failure.	All services plus full-resolution logging.
Mode transitions	No old action accepted after DETOUR or RESUME.	Epochs, timestamps and injected late outputs.

```
sudo nvpmode1 -q
tegrastats --interval 1000 --logfile orin_stats.txt
```

Measure each mode and the transitions

Benchmark the classical comparator and then the complete unified system. Warm up and collect at least 1,000 model decisions across survey, inspection and analysis, with mode counts reported. Include cold start and rare transition peaks separately. Report p50, p95, maximum latency, missed deadlines, RAM, power mode and temperature. [35, 36]

Include image preprocessing, grounding, all active heads, projection, logging and manager work in end-to-end latency. A GPU kernel time alone cannot support a real-time claim. Use real-time simulator pacing when coupling the Orin to SITL.

Deliverable: a trace connecting image capture, event/action output, accepted manager transition, setpoint send and vehicle response. If the model fails the gate, reduce input demand or flight speed and repeat the mission test; retain the original failed benchmark in the report.

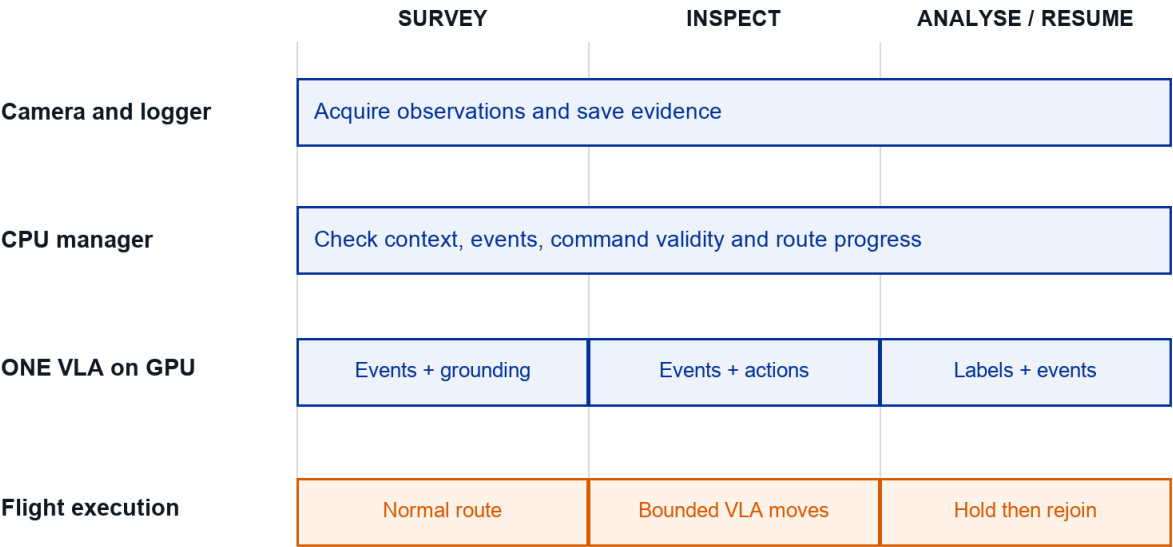
CHAPTER 6 REASONING AND ONBOARD COMPUTE

Schedule one model across mission modes

The same model is invoked throughout the mission, but its required outputs change. Route following, logging and command validation remain independent of model inference.

ONE MODEL ACROSS MISSION MODES

Phases are schematic; durations and achievable rates must be measured.



The model decides when to detour and resume. The manager preserves unfinished coverage.

Figure 11. Proposed schedule for one VLA. Widths represent mission phases, not measured durations or achievable rates.

Avoid hidden compute or control changes

Use event and grounding outputs while surveying, continuous actions while inspecting, and semantic outputs during a held analysis. Log what executed at each phase. The survey executor resumes only after a validated RESUME event or a separately recorded override.

Benchmark the transitions

Measure observation gaps during survey and command age during inspection. Include analysis holds and rejoin travel in mission time. A single resident model can reduce duplicated weights, but its peak memory and latency still require measurement.

CHAPTER 7 TRANSFER AND RESEARCH EVALUATION

Define the sim to real gap precisely

The sim to real gap is a difference in performance under a specified comparison. It is not a single visual realism score. Measure the mechanisms that can change the final mission, then test the integrated system.

Source of gap	What changes	Measurement to collect
Appearance	Grass, texture, light, shadows, object instances.	Detection recall and VLA success by condition.
Camera	Intrinsics, blur, exposure, compression, rolling shutter.	Calibration residuals, pixel sizes and image-quality distributions.
State estimation	GNSS bias, altitude error, attitude noise, time offsets.	Reference point residuals and pose/timestamp uncertainty.
Motion	Command response, wind, drift and stopping distance.	Step response and tracking traces under approved scripted commands.
Compute and communication	Inference delay, drops, queues and thermal throttling.	Capture-to-command age, tail latency and missed deadlines.
Mission context	Field shape, item density and occlusions.	Condition-matched search and inspection results.

Calibrate the simulator using development data

Record short real camera sessions and scripted motion traces before final training. Match image scale and camera geometry first. Estimate plausible appearance, pose noise, delay and command-response ranges. Use these to set simulation parameters and randomisation [34]. Keep the real test sessions untouched; tuning the simulator on the test field outcomes contaminates the transfer measurement.

What you do not need to develop

Keep the autopilot controller fixed. Adjust the simulated vehicle and sensor configuration to approximate the measured response where supported. For early skill development, a simple command-response approximation may help diagnosis, but final simulated flight evaluation should use the chosen SITL stack. Do not claim identified full vehicle dynamics from a few short traces.

State the comparison domain

Match category mix, target pixel size, starting offset, altitude, flight speed and lighting bands as closely as practical. Keep genuinely novel conditions in a separate stress test. If simulated fields are easier than real fields, the observed gap includes task difficulty as well as transfer; explain that rather than attributing everything to visual domain shift.

CHAPTER 7 TRANSFER AND RESEARCH EVALUATION

Evaluate transfer in five stages

Stage 1 Real images without model control

Evaluate the unified VLA grounding and semantic outputs on held-out real imagery. Check geometric projection against independently surveyed objects. The VLA may produce actions in replay for inspection, but these actions did not change the recorded future images. Action agreement in replay is only a diagnostic, not real flight success.

Stage 2 Real device with simulated motion

Run the Orin against real-time SITL using the deployment protocol on page 60. Add observed communication delays, missing frames and model stalls. Verify that queue limits and the watchdog behave correctly. Report this stage separately from both workstation-only simulation and real flight.

Stage 3 Scripted real commands and shadow inference

Under the lab’s flight procedure, have the established autopilot and pilot execute a short scripted inspection. Record tracking response, image quality and latency. Run the VLA in shadow mode, logging proposals without giving it control. Compare its proposed direction with the visible target and detect frame or scale errors before enabling it.

Stage 4 Restricted real skill trials

Start with one visible object, then introduce category choice. Use the same fixed altitude, yaw, bounded region and limited motion as the validated task. A trained pilot can override immediately. Progress only after the lab accepts the previous stage. Run the classical and learned policies on comparable starts and objects, randomising order where practical.

Stage 5 Complete field missions

Evaluate search, inspection, analysis and mapping together under a fixed mission time. Alternate methods within comparable field sessions and record environmental changes. Count all assigned trials, including aborted missions and interventions. Record how much field operation was autonomous and which components used external compute.

Evidence obtained	Claim it can support
Real imagery and surveyed mapping only	Transfer of perception and geometric mapping under those conditions.
Orin plus SITL	Deployment feasibility and timing with simulated dynamics.
Restricted real learned-policy trials	Transfer of the tested local inspection skill.
Full real missions	Transfer of the integrated mission within the tested envelope.

If a stage fails, diagnose it before expanding the envelope. Field-access or flight limitations should reduce the claim, not disappear from the thesis. Check the applicable operating requirements with the lab and current official guidance [26].

CHAPTER 7 TRANSFER AND RESEARCH EVALUATION

Design a transfer comparison you can interpret

Use the following comparison to distinguish transfer from adaptation. Keep model architecture, action contract and evaluation scenes fixed wherever possible. Save the exact datasets used by each row.

Training variant	Simulated test	Real test
S0 Nominal simulation only	Establish a nominal baseline.	Zero-shot transfer; no real labels used for weights.
S1 Randomised simulation	Measure robustness within simulation.	Test whether plausible variation helps.
S2 S1 plus real development demonstrations	Check for loss of simulated performance.	Measure the value of limited real adaptation.
Classical baseline	Same geometry, route and command limits; its own detector.	Same available inputs and field conditions.

If S1 randomisation was calibrated using real development images or logs, say so: it has no real action-label training but is not a no-real-data experiment. For S2, plot real performance versus the number of adaptation episodes. Never adapt on the locked test sessions.

Report absolute gaps with uncertainty

$$\Delta = 100 \times (s - r)$$

Here s and r are simulated and real success fractions between 0 and 1; Δ is the gap in percentage points.

For error metrics, report real minus simulated error in the original units. For example, 85% simulated and 60% real success imply a 25 percentage-point gap. These are illustrative values. Always report both values, trial counts and confidence intervals.

Use episodes for skill tests and missions for mission tests. Estimate uncertainty across independent layouts or sessions, not neighbouring frames. Analyse paired differences where possible. For different physical scenes, match condition groups and explain the limitation.

Start with a feasible sample plan

A proposed simulation plan is 30 held-out layouts per main condition and three training seeds when affordable. Begin real development with five trials per method. A candidate final plan is 12 held-out layouts across two or more sessions, pairing methods per layout where feasible. Choose the final count using development variability and field access; this is a planning budget, not a power calculation. Small field samples justify descriptive results and wide intervals, not claims of universal transfer.

Use diagnostic ablations

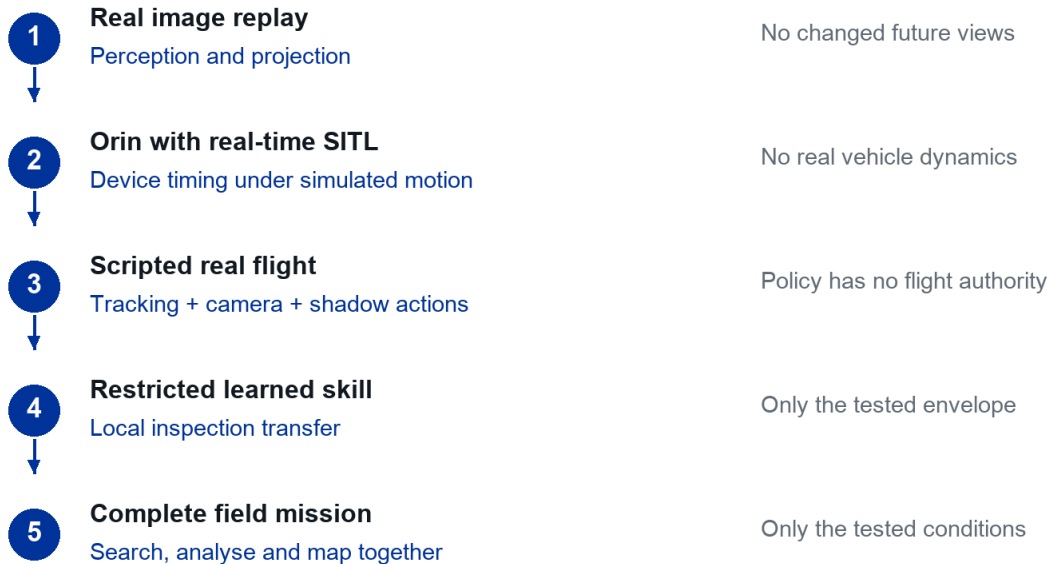
Change one simulated factor at a time: appearance, pose noise, latency and command-response mismatch. Then test the combined condition. Compare those failure patterns with real traces. Oracle pose or target identity can localise the problem, but must be labelled as privileged diagnostics rather than deployable performance.

CHAPTER 7 TRANSFER AND RESEARCH EVALUATION

Match the transfer claim to the evidence

Testing a model on real images is useful, but it does not show what future images its actions would create. Each stage below addresses a different part of that gap.

INCREASE THE STRENGTH OF THE EVIDENCE IN STAGES



At every stage: record failures, interventions, conditions and the claim the evidence supports.

Figure 12. The staged transfer protocol. Each stage adds evidence while retaining the limitations shown on the right.

Advance on evidence

Use each stage to diagnose appearance, geometry, timing and command-response errors before allowing more autonomy. Record the same task definition, action schema and failure reasons so stages remain interpretable.

A valid reduced thesis scope

If the project reaches Orin with SITL and real-image replay only, report device feasibility and perception transfer. A claim about real learned navigation requires the relevant closed-loop flight evidence.

CHAPTER 7 TRANSFER AND RESEARCH EVALUATION

Score the mission without hiding failures

Choose a primary metric before final experiments. Recommended: the fraction of all placed litter items correctly located and categorised by a fixed mission deadline. Show map recall without the category requirement alongside it, so perception and analysis errors remain distinguishable.

Metric	Definition or reporting rule
Inspection success	Correct requested item centred with the stated dwell, altitude and speed conditions; failures stay in the denominator.
Map recall	Unique ground-truth items matched to valid map predictions divided by all eligible placed items.
Map precision	Matched predictions divided by all predictions; unmatched predictions include false positives and duplicates.
Correctly analysed recall	Matched items with correct category divided by all eligible placed items.
Location error	Median and 95th percentile distance of matched pairs, plus the number of pairs.
Efficiency	Time to a declared recall level, or recall over time, under the same time budget.
Robustness	Interventions, rejected/stale commands, timeouts and task failures per trial.
Compute	Latency distribution, deadline misses, peak memory and sustained power/thermal observations.

Match predictions to truth once

Define a spatial gate in metres using the intended mapping accuracy and ground-truth uncertainty. Within that gate, find a maximum-cardinality one-to-one matching, then minimise total distance among equally sized matchings. Include unmatched options. Do not perform unconstrained nearest assignment and only remove distant pairs afterwards; it can miss valid alternatives.

Use class-agnostic spatial matching for map recall, then check category correctness on the matched pairs. This prevents wrong labels from disappearing into a different matching rule. Publish the gate, coordinate reference, uncertain ground-truth cases and duplicate definition.

Define the mission clock

Choose whether the budget begins at takeoff or at survey start, and use the same definition for every method. Include inspection, model analysis, resumption travel, waits and recovery. Report battery or power effects separately if measured. Avoid a score that multiplies recall by $1 - \text{elapsed_time} / \text{budget}$: it becomes zero at the deadline even for a successful completed mission.

Count interruption and resumption failures

Report unnecessary detours, inspection yield, repeated inspections, resumption success, skipped coverage and rejoin time. Resumption must continue unfinished coverage without stale actions. Use reviewed visible opportunities for event diagnostics and all placed objects for mission recall. Save a primary cause and supporting trace for every failure.

CHAPTER 7 TRANSFER AND RESEARCH EVALUATION

Compare the single VLA with fair baselines

ID	System	Purpose
B0	Tuned detector, priority rules and classical inspection.	Practical end-to-end classical baseline.
B1	Unified model perception/labels/actions, but fixed event rules.	Isolate the learned decision contribution.
U	One VLA predicts events, regions, labels and movements.	Target system.
U without context	Same model with mission history removed or masked.	Test the contribution of coverage and object memory.

Keep the route, flight limits, camera, geometry, association code, field layouts and mission deadline fixed. B0 and U use different perception components; their difference is a system comparison. B1 and U should share the same visual evidence, weights outside the event decision and movement execution so the decision comparison is interpretable.

Add targeted ablations after the main result

Compare fixed rules against learned events using the same checkpoint. Disable additional views to test active inspection. Compare a classical local executor using the model's grounded region against its action head. Test nominal simulation, randomised simulation and limited real adaptation separately. Avoid an unmanageable Cartesian product of all variants.

Make language measurable

Use instructions that alter the task, such as prioritising bottles versus cans, and include different valid wording. Pair scenes while changing the instruction. If every trial uses "find all litter", do not claim general instruction following from success alone.

Freeze the protocol and analyse paired missions

Select checkpoints and thresholds on development data. Fix metric, matching gate, event rules, timeout, seeds and test manifests before final runs. Run methods on paired simulated layouts. Alternate field methods and block by session; confidence intervals must reflect independent layouts/sessions, not adjacent frames.

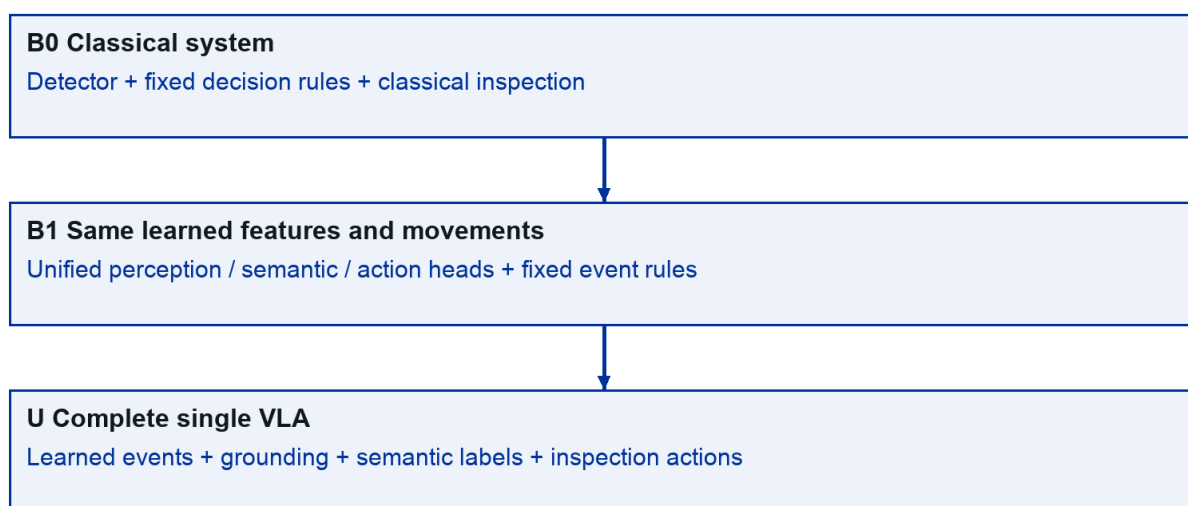
RQ1 needs complete mission evidence and correct resumption. RQ2 needs grounding, movement and evidence ablations. RQ3 needs actual board measurements and staged transfer. Report negative outcomes and rejected/overridden events; they are part of the method's cost.

CHAPTER 7 TRANSFER AND RESEARCH EVALUATION

Identify what makes the mission improve

Use a practical system baseline and a comparison that holds the learned perception and movement components constant. This separates system-level benefit from the value of learned mission decisions.

WHAT DOES THE UNIFIED POLICY CONTRIBUTE?



B0 versus U: complete system value

B1 versus U: value of learned mission decisions

Also remove mission context to test the contribution of history and coverage.

Figure 13. Main comparisons for the single VLA thesis. These are experiment definitions, not measured performance.

Read each comparison as a question

B0 versus U asks whether the complete learned system improves the application. B1 versus U asks whether learned events help when the same model supplies perception, labels and actions. U without context tests whether mission memory contributes.

Keep the comparison executable

Use the same coverage route, clock, flight envelope, geometry and map matching. Count all waits, failures and overrides. Add further ablations only after these complete-mission comparisons are reliable.

CHAPTER 8 PROJECT PLAN AND THESIS WRITING

A thesis plan with support and exit criteria

Weeks	Primary work	Exit evidence and support
1–2	Laptop exercises and foundations	Reproduce baseline, train and explain failures; supervisor checks understanding
3–5	PX4, camera and command interface	Square, axis checks and synchronised bridge; lab engineer leads integration review
6–8	Complete classical mission and dataset pilot	Search, inspect and map ten objects; supervisor approves dataset schema
9–12	Simulation data and VLA adaptation	GPU smoke test, learning curves and language tests; ML support resolves model setup
13–16	Unified mission heads and Orin profiling	Rule versus learned events, resume tests and runtime trace
17–19	Calibration and real development data	Measured appearance, geometry and latency ranges; pilot and lab engineer support
20–22	Frozen simulated and real evaluation	Trial ledger, confidence intervals and failure taxonomy
23–26	Analysis, writing and reproducibility	Re-run instructions, dataset card, model card and defensible conclusions

Define scope reductions before you need them

If VLA inference is too slow, keep it as an offline or simulator comparison and deploy the classical baseline. If real flights are unavailable, report a simulator and hardware-inference study and explicitly leave closed-loop real transfer unproven. If learned mission decisions add no benefit, report the comparison with B1. If only movement is learned, label the outcome a partial thesis scope; it does not establish the proposed unified system.

Use a short weekly supervision note

Record: the question tested, the command or protocol, one result figure, one failure, the interpretation and the next experiment. Bring a specific blocker such as “camera capture timestamps differ from pose timestamps by an unknown offset”, not only “the simulator does not work”.

Advance when the stated evidence is ready. Installation problems, calibration and model adaptation need buffer time; discuss delays early and revise the scope with the supervisor.

CHAPTER 8 PROJECT PLAN AND THESIS WRITING

Diagnose the symptom before changing the model

Symptom	First check	Evidence that resolves it
Moves away from the target	Axis signs, image orientation and action frame	North/east marker experiment
Moves farther on every resend	Delta incorrectly added repeatedly	Constant accepted absolute target in log
Map shifts during flight	Pose-image clock alignment and camera extrinsics	Known marker projected at several speeds
Training loss improves but missions fail	Distribution drift or mismatched action scale	Failure replay with expert action overlay
Language change has no effect	Target leaked through crop or missing task input	Same-scene different-task trial
Objects are missed at survey height	Pixel size, motion blur and candidate threshold	Recall plotted against object pixel width
Orin occasionally freezes	Memory pressure, thermals or competing inference	Concurrent RAM, temperature and latency trace
Many map duplicates	Weak association or repeated frame-level entries	Track history with evidence IDs

Terms you should be able to explain

Policy: a function from observations to actions. **Planner:** chooses a task or route. **Controller:** tracks the chosen target. **SITL:** autopilot software connected to a simulated vehicle. **VLM:** a model using images and language. **VLA:** a model using visual and language inputs to predict actions. **Behavioural cloning:** supervised learning from expert actions. **Ground truth:** reference information used to label or score results. **Ablation:** remove or change one component to test its contribution.

Know when to request help

Spend a focused session isolating a bug, then bring the smallest reproducible command, the full error, the environment version and what you expected. Coordinate transforms, time synchronisation and flight failsafes deserve review even when the code appears to run.

CHAPTER 8 PROJECT PLAN AND THESIS WRITING

Build the thesis around the research questions

The thesis manuscript should explain a research argument. Use the labs as evidence of implementation and understanding, then organise the written chapters around the problem, method and measured result. Replace proposed settings with the actual values used in the final experiments.

Thesis chapter	Question it answers	Evidence you should already have
1 Introduction	Why this problem and what exactly is tested	Scope, RQ1–RQ3 and defined contributions
2 Related work	What prior work establishes and what remains open	Comparison matrix and paper notes
3 System and simulator	How observations become safe position commands	Architecture, frame contract and simulator validation
4 Data and learning	What data trained which component and how	Dataset card, split manifest and training records
5 Experimental protocol	How methods are compared fairly	Baselines, frozen metrics and trial manifests
6 Results	What happened in each experiment	Tables, paired plots, failures and uncertainty
7 Deployment and transfer	What changes on Orin and in the real field	Runtime traces and staged transfer results
8 Discussion and conclusion	What is supported, limited and still unknown	Answers to RQ1–RQ3 with bounded claims

Write while the evidence is fresh

Draft the system chapter after the classical mission works, the data chapter after the pilot audit, and the protocol before the final tests. Add results only from saved runs. Leave clear placeholders for experiments not performed instead of turning planned behaviour into a past-tense claim.

Link every contribution to an artifact

A contribution such as “a reproducible aerial inspection dataset” needs a schema, collection procedure, splits and access conditions. A contribution such as “efficient onboard VLA inference” needs actual device timing and memory under the full workload. An architecture diagram alone cannot establish either claim.

CHAPTER 8 PROJECT PLAN AND THESIS WRITING

Write the introduction and related work

Start with the actual task

Describe the field, deliberately placed litter, available sensing and expected map. Explain why finding an object, obtaining an informative view and estimating its location are different decisions. Introduce the Orin resource constraint and the use of an existing autopilot before discussing model families.

Make the research gap conditional and specific

Use the literature to establish what the baseline already solves. Then identify the comparison this thesis will perform: for example, whether one VLA chooses useful survey interruptions and inspection movements under a fixed budget while preserving coverage and avoiding duplicate objects. Avoid a broad statement that existing drones cannot reason or that no related system exists.

A paragraph structure you can reuse

Prior work establishes X under conditions Y [citations].
Our field task differs in A, B and C.
We therefore evaluate method M against baseline B
using metric Q and a held-out protocol.
This tests the stated difference; it does not assume a gain.

Synthesize rather than catalogue

Group related work by aerial perception and mapping, action policies, reasoning and active perception, then transfer and compute. Compare inputs, outputs, training data, environment and evaluation. Explain how each group informed one implementation or experimental decision. Keep model descriptions that do not affect the project short.

Keep an auditable reading record

Record the query, search date, source, inclusion reason, publication status and version. Read the experiment section before quoting performance. Distinguish an abstract's broad claim from the narrower evidence in its tables. Prefer the original paper, dataset release and official implementation over a secondary summary.

Draft the introduction last again

An early introduction sets scope. Revise it after results are known so the contributions describe completed work. If real closed-loop flight was not completed, say that the evaluation comprises simulation, device profiling and real-image replay; do not promise real navigation in the final abstract.

CHAPTER 8 PROJECT PLAN AND THESIS WRITING

Write methods another student can reproduce

A methods chapter should let another student reconstruct the experiment without guessing units, versions or split rules. Put implementation details in the main text when they affect the result; place long configuration listings in an appendix or the versioned code supplement.

Describe one observation and one action precisely

State the camera resolution, calibration, frame conventions, timestamps, state features, language and mission-context input, event vocabulary, grounding fields, per-head masks and action dimensions. Explain capture anchoring, physical scaling, expiry, fixed altitude and yaw, and the role of the existing autopilot. Include a worked record from the logs and separate deployable measurements from privileged evaluator truth.

Describe the data before reporting model size

Report collection domains, physical or simulated object instances, layouts, acquisition sessions and exclusions. Give episode and frame counts per split. Explain grouping and show that calibration, threshold selection and fine-tuning never use the locked test set. Include annotation rules for uncertain and absent objects.

Describe every trainable choice

Name the pretrained checkpoint and revision; identify reset projections, frozen weights and trainable modules. Record objective, optimiser, learning rate, batch size, precision, number of updates, data sampling and checkpoint criterion. The provided pilot script selects validation flow loss; a research run may use a declared closed-loop validation rule. State which was used.

Make baselines comparable

Describe the detector-based inspection rule, the planning heuristic and the common map code. List components held fixed across comparisons. Where a method changes runtime load, include that timing in the mission. Label oracle experiments and keep them separate from deployable results.

A compact methods checklist

For each result table, a reader should find: one code revision, one environment record, one dataset manifest, one checkpoint or prompt, one action schema, one evaluation command and one metric definition. A random seed without the underlying assets and versions is insufficient for reproduction.

CHAPTER 8 PROJECT PLAN AND THESIS WRITING

Write results and discussion from saved runs

Answer each research question with a result, its uncertainty and a failure explanation. Keep observations separate from interpretations. A table of successful demonstrations alone cannot explain which component helped or how often it failed.

Result family	Useful figure or table	Interpretation to include
Local inspection	Paired success by scene and target; action traces	Correct target, stability, failure causes and language dependence
Whole mission	Map recall over time; precision and duplicate rate	Coverage, perception, revisit cost and association errors
Learning	Validation curves and data-size comparisons	Overfitting, label quality and learner distribution shift
Orin runtime	Latency distribution aligned with RAM and temperature	Deadlines, contention, sustained performance and fallback
Transfer	Simulated and real values with paired differences	Which mismatch explains the change and which remains uncertain

Use the educational result correctly

The supplied toy learner succeeded in 24 of 30 held-out trials, compared with 30 of 30 for the classical colour baseline. This demonstrates how to report an unfavourable learned-policy comparison. It is teaching evidence from this package, not an original drone result produced by the student. Reproduce it if useful, label its origin and keep it outside claims about real litter navigation.

Discuss limitations at the level of the evidence

Distinguish limited object diversity, one field site, uncertain ground truth, unmeasured timing and unavailable real flights. Explain the likely effect of each limitation. Do not write that a model “generalises well” when only new seeds of the same toy scene were tested.

Conclude without hiding a negative result

State what the best method achieved, under which conditions, at what resource cost, and where it did not help. A finding that geometry and a detector outperform a VLA can be scientifically useful when the comparison is fair and its causes are analysed. Propose future work tied to those observed causes.

CHAPTER 8 PROJECT PLAN AND THESIS WRITING

Build a publishable contribution

A successful prototype can support a paper when it establishes a defensible contribution with reproducible evidence. A working demonstration alone does not establish novelty or comparative value. Publication and venue suitability remain reviewer judgements.

Position the question against existing work

Aerial litter detection and mapping already have a reference in UAVVaste [1]. CognitiveDrone [7] includes aerial actions and a separate-reasoner variant. pi0.5 [44] provides precedent for unified high-level and continuous outputs. AerialVLA navigation work also exists [45, 46]. Therefore, “a VLA on a drone” and “one model for decisions and actions” are insufficient novelty claims by themselves.

Candidate contribution: a compact single-model method and evaluation protocol for interruptible aerial litter surveying, where useful inspections must improve the map without losing unfinished coverage. Validate this positioning with a deeper comparison of methods, code and experiments before submission; the source check here is not an exhaustive novelty search.

Evidence that would make the paper persuasive

- A precise method for joint mission events, grounding, semantic evidence and bounded actions, with training masks and failure handling.
- A complete-mission comparison against B0 and B1 under equal time and flight constraints, plus focused ablations that identify the source of any gain.
- Held-out layouts, physical objects and field sessions; uncertainty estimates across independent trials and actual training-seed counts.
- Measured Orin execution and controlled real-flight missions, with rejected events, false detours, duplicate maps and resumption failures retained.
- A reusable task definition, split manifest, evaluation code and releasable data/evidence, including failures.

Choose the claim to match the result

A sound real-world systems study could be a full robotics or aerial-systems paper. A limited simulator prototype may suit a workshop or preliminary report. A negative result can be useful if the benchmark, causal comparisons or deployment findings teach something new. Avoid promising acceptance or presenting planning counts as a power calculation.

CHAPTER 9 RESEARCH RECORDS AND GLOSSARY

Templates for supervision and field data

Weekly meeting note

Week and date: _____ Current milestone: _____
 Question I worked on: _____
 Result figure and run ID: _____
 What failed and what I learned: _____
 Decision or help needed: _____
 Next deliverable and completion test: _____

Field session record

Session ID and date: _____
 Site and coordinate origin: _____
 Pilot and collection method: _____
 Camera settings and calibration ID: _____
 Object inventory and survey uncertainty: _____
 Weather, ground and illumination: _____
 Methods, starts and order of trials: _____
 Data split and physical instance IDs: _____
 Dropped data, overrides and unusual events: _____
 Raw data location and backup check: _____

Paper note

Citation and link: _____
 Task and environment: _____
 Inputs and outputs, including units: _____
 Training data and learning method: _____
 Baselines, test split and metrics: _____
 Result with its evaluation context: _____
 Limitation for this project: _____
 One idea to use or test: _____

Definition of done

I can reproduce the result from a saved configuration. I inspected successes and failures. I can explain the action units, split and metric. The supervisor can see the output without running an undocumented notebook.

CHAPTER 9 RESEARCH RECORDS AND GLOSSARY

Templates for experiments and reporting

Experiment record

Run ID / date / code commit: _____
 Research question and compared methods: _____
 Changed variable and fixed components: _____
 Dataset / split / layout IDs / seeds: _____
 Checkpoint / prompt / calibration: _____
 Hardware / runtime / power mode: _____
 Primary metric and success threshold: _____
 Failures, interventions and exclusions: _____
 Output files and conclusion: _____

Orin benchmark record

Carrier / cooling / storage / JetPack: _____
 Model revisions / precision / input sizes: _____
 Workload and duration after warm-up: _____
 Capture-to-command p50 / p95 / p99: _____
 Deadline misses / peak RAM / temperature: _____
 Power measurement and mode: _____
 Decision latency / analysis hold / override count: _____
 Deployment decision and reason: _____

Claim and evidence record

Claim I intend to make: _____
 Research question and experiment ID: _____
 Primary result with denominator and uncertainty: _____
 Alternative explanation or limitation: _____
 Figure and source run files: _____
 What this evidence does not establish: _____

Final reproducibility package

Include environment manifests; simulator assets and seeds; permitted data or retrieval instructions; annotation rules; grouped splits; model and normaliser versions; all prompts; a flight-adaptor specification; evaluation commands; and scripts that recreate the figures. Label example configurations and measured final configurations clearly.

CHAPTER 9 RESEARCH RECORDS AND GLOSSARY

Terms to keep beside the simulator

Term	Meaning in this guide
VLM	Model conditioned on vision and language, used for analysis or task decisions.
VLA	Learned vision and language conditioned policy that predicts robot actions.
VLN	Vision and language navigation to goals described in words.
Episode	One trial from a defined start to completion, failure or timeout.
Rollout	Running a policy through successive observations and actions.
Epoch	One pass through the selected training samples.
Imitation learning	Learning action behaviour from expert demonstrations.
Flow matching	A training method for generating continuous outputs from noise; can be used with supervised action demonstrations.
RL	Reinforcement learning; a policy is improved using rewards and experience.
Action chunk	Several future actions predicted together, with explicit timing.
Grounding	Associating a word or instruction with the relevant visible object or place.
Active perception	Choosing motion or viewpoints to acquire missing evidence.
GSD	Ground sampling distance; ground size represented by one image pixel.
Extrinsics	Position and orientation of one sensor or frame relative to another.
GNSS and RTK	Satellite positioning; RTK uses correction data to improve positioning under suitable conditions.
Domain randomisation	Varying documented simulation conditions during training.
Zero-shot transfer	Testing a frozen policy in the target domain without target-domain weight adaptation; disclose any real-data calibration.
Ablation	Controlled experiment removing or changing one component.
p95 latency	A latency value at or below which 95% of measured requests fall.
Intervention	A pilot or supervisor takes over or a fallback ends the assigned learned task.

BIBLIOGRAPHY

References for robotics and litter

References are numbered consistently throughout this document. Linked preprints are distinguished from conference and journal records. Preserve the version actually used in the thesis.

[1] Kraft, M., Piechocki, M., Ptak, B., and Walas, K. (2021). Autonomous, Onboard Vision-Based Trash and Litter Detection in Low Altitude Aerial Images Collected by an Unmanned Aerial Vehicle. *Remote Sensing* 13(5), 965.

[doi:10.3390/rs13050965](https://doi.org/10.3390/rs13050965)

[2] Bartolo, M., et al. (2026). Litter detection from aerial imagery: a review of UAV-based approaches and deep learning techniques. *Multimedia Tools and Applications* 85, article 232.

[doi:10.1007/s11042-026-21440-1](https://doi.org/10.1007/s11042-026-21440-1)

[3] Sa, I., Park, C., Lee, H.-M., Noh, D., and Ahn, H. S. (2026). Vision-Language-Action (VLA) Models for Unmanned Aerial Robotics and Bimanual Manipulation: A Review. *Drones* 10(6), 412.

[doi:10.3390/drones10060412](https://doi.org/10.3390/drones10060412)

[4] Brohan, A., et al. (2023). RT-2: Vision-Language-Action Models Transfer Web Knowledge to Robotic Control. Research paper; linked arXiv record.

[arXiv:2307.15818](https://arxiv.org/abs/2307.15818)

[5] Kim, M. J., et al. (2024). OpenVLA: An Open-Source Vision-Language-Action Model. Research paper; linked arXiv record.

[arXiv:2406.09246](https://arxiv.org/abs/2406.09246)

[6] Shukor, M., et al. (2025). SmolVLA: A Vision-Language-Action Model for Affordable and Efficient Robotics. Research paper; linked arXiv record.

[arXiv:2506.01844](https://arxiv.org/abs/2506.01844)

[7] Lykov, A., et al. (2025). CognitiveDrone: A VLA Model and Evaluation Benchmark for Real-Time Cognitive Task Solving and Reasoning in UAVs. Research paper; linked arXiv record.

[arXiv:2503.01378](https://arxiv.org/abs/2503.01378)

[8] Hu, C. Y., et al. (2025). See, Point, Fly: A Learning-Free VLM Framework for Universal Unmanned Aerial Navigation. *CoRL*; PMLR 305, 4697-4708.

[CoRL paper and code links](#)

[9] Zawalski, M., et al. (2024). Robotic Control via Embodied Chain-of-Thought Reasoning. Research paper; linked arXiv record.

[arXiv:2407.08693](https://arxiv.org/abs/2407.08693)

BIBLIOGRAPHY

References for reasoning and aerial tasks

[10] Yin, C., et al. (2025; revised 2026). DeepThinkVLA: Enhancing Reasoning Capability of Vision-Language-Action Models. Version 3, 5 August 2026.

[arXiv:2511.15669v3](https://arxiv.org/abs/2511.15669v3)

[11] Yao, S., et al. (2022; ICLR 2023). ReAct: Synergizing Reasoning and Acting in Language Models. Conference paper; linked arXiv record.

[arXiv:2210.03629](https://arxiv.org/abs/2210.03629)

[12] Cai, H., et al. (2025). FlightGPT: Towards Generalizable and Interpretable UAV Vision-and-Language Navigation with Vision-Language Models. EMNLP 2025.

[ACL Anthology publication record](#)

[13] Cheng, A.-C., et al. (2024; revised 2025). NaVILA: Legged Robot Vision-Language-Action Model for Navigation. Research paper; linked arXiv record.

[arXiv:2412.04453](https://arxiv.org/abs/2412.04453)

[14] Zhang, D., Chen, P., Zhou, J., and Yang, S. (2026). ESARBench: A Benchmark for Agentic UAV Embodied Search and Rescue. Preprint.

[arXiv:2605.01371](https://arxiv.org/abs/2605.01371)

[15] Zhang, W., et al. (2026). ActiveFly-Bench: Aligning Embodied Question Answering with Vision-Language-Action for Aerial Embodied Perception. Preprint.

[arXiv:2607.10180](https://arxiv.org/abs/2607.10180)

[16] Lu, W., et al. (2026). ScoutVLA: UAV-Centric Active Perception via a Dual-Expert VLA Model for Open-World Embodied Question Answering. Preprint.

[arXiv:2606.14772](https://arxiv.org/abs/2606.14772)

[17] Zheng, X., et al. (2026). Think Like a Pilot: Fine-Grained Long-Horizon UAV Navigation. Preprint introducing FLIGHT and FLIGHT VLA.

[arXiv:2606.06836](https://arxiv.org/abs/2606.06836)

[18] Black, K., et al. (2024; RSS 2025). pi0: A Vision-Language-Action Flow Model for General Robot Control. Conference paper; linked arXiv record.

[arXiv:2410.24164](https://arxiv.org/abs/2410.24164)

BIBLIOGRAPHY

References for perception and flight interfaces

[19] Proenca, P. F., and Simoes, P. (2020). TACO: Trash Annotations in Context for Litter Detection. Dataset paper; linked arXiv record.

[arXiv:2003.06975](https://arxiv.org/abs/2003.06975)

[20] Liu, S., et al. (2023; ECCV 2024). Grounding DINO: Marrying DINO with Grounded Pre-Training for Open-Set Object Detection. Conference paper; linked arXiv record.

[arXiv:2303.05499](https://arxiv.org/abs/2303.05499)

[21] Bai, S., et al. (2025). Qwen2.5-VL Technical Report. Technical report.

[arXiv:2502.13923](https://arxiv.org/abs/2502.13923)

[22] Cabreira, T. M., Brisolara, L. B., and Ferreira Jr., P. R. (2019). Survey on Coverage Path Planning with Unmanned Aerial Vehicles. Drones 3(1), 4.

[doi:10.3390/drones3010004](https://doi.org/10.3390/drones3010004)

[23] Hugging Face LeRobot (Living documentation). SmolVLA. Training, dataset and deployment guidance.

[Official SmolVLA documentation](#)

[24] MAVSDK (Living documentation). Offboard Control. Setpoint interface and automatic resending behaviour.

[Official offboard API guide](#)

[25] PX4 (Living documentation). Offboard Mode. Official user-guide source; proof-of-life and failsafe behaviour.

[Official PX4 user guide source](#)

[26] EASA (Living guidance). Open Category Low Risk Civil Drones. Start here, then verify local rules and operation-specific requirements.

[EASA operating-category guidance](#)

[27] Butler, H., et al. (2016). The GeoJSON Format. IETF RFC 7946.

[RFC 7946 standard](#)

[28] OpenCV (Version 4 documentation). Camera Calibration and 3D Reconstruction. Intrinsic, distortion and geometric projection.

[Official calibration reference](#)

BIBLIOGRAPHY

References for simulation and learning

[29] **PX4 (Version 1.16 documentation)**. Ubuntu Development Environment. Official installation guide for the worked simulator stack.

[PX4 installation](#)

[30] **PX4 (Version 1.16 documentation)**. Gazebo Simulation. Launch commands and Gazebo Harmonic integration.

[PX4 Gazebo guide](#)

[31] **Gazebo (Transport 13 documentation)**. Python Support. Transport subscriptions and publishing.

[Gazebo Transport Python](#)

[32] **MAVSDK (Living documentation)**. Python QuickStart. Connection and example applications.

[MAVSDK Python guide](#)

[33] **Ross, S., Gordon, G., and Bagnell, D. (2011)**. A Reduction of Imitation Learning and Structured Prediction to No-Regret Online Learning. DAgger; AISTATS.

[DAgger paper](#)

[34] **Tobin, J., et al. (2017)**. Domain Randomization for Transferring Deep Neural Networks from Simulation to the Real World. Domain randomisation research paper.

[Domain randomisation paper](#)

[35] **NVIDIA (2025)**. JetPack 6.2 Brings Super Mode to Jetson Orin Nano and Jetson Orin NX Modules. Power modes require the supported software and hardware configuration.

[NVIDIA JetPack 6.2 technical article](#)

BIBLIOGRAPHY

References for deployment and implementations

[36] **NVIDIA (Jetson Linux R36.4.3 documentation)**. Tegrastats Utility. System resource and thermal monitoring.

[Tegrastats reference](#)

[37] **Hugging Face (Model card)**. SmolVLM2 500M Video Instruct. Candidate small visual language model; device performance must be measured.

[Official model card](#)

[38] **Vrije Universiteit Brussel (House style resources)**. Visual identity. Institutional colour and visual identity reference.

[VUB visual identity](#)

[39] **Hugging Face LeRobot (Model card)**. SmolVLA base. Checkpoint, training objective and intended fine-tuning use.

[Official SmolVLA model card](#)

[40] **Gazebo (Harmonic documentation)**. Sensors. Sensor configuration and links to examples.

[Gazebo sensor guide](#)

[41] **NVIDIA (Living documentation)**. Installing PyTorch for Jetson Platform. Select packages compatible with the installed JetPack release.

[NVIDIA Jetson PyTorch guide](#)

[42] **Hugging Face LeRobot (Version v0.4.4)**. Source code for the supplied SmolVLA integration. Policy, processor, dataset and factory implementation.

[Pinned LeRobot source](#)

[43] **MAVSDK (Version 2.8.4)**. MAVSDK Python package. Pinned API for the supplied SITL square exercise.

[Pinned MAVSDK release](#)

BIBLIOGRAPHY

References for unified models and aerial comparisons

[44] Physical Intelligence et al. (2025). pi0.5 A Vision Language Action Model with Open World Generalization. arXiv:2504.16054. Unified high-level and continuous outputs; manipulation evidence.
[pi0.5 paper](#)

[45] Xu et al. (2026). AerialVLA A Vision Language Action Model for UAV Navigation via Minimalist End to End Control. arXiv:2603.14363. Navigation preprint; distinguish from the identically named AAAI paper.
[AerialVLA navigation preprint](#)

[46] Chen et al. (2026). AerialVLA A Vision Language Action Model for Aerial Navigation with Online Dialogue. AAAI 40(22), 18161–18169. DOI 10.1609/aaai.v40i22.38878.
[AAAI AerialVLA paper](#)

[47] Mehboob et al. (2026). DroneVLA VLA based Aerial Manipulation. arXiv:2601.13809. Related aerial manipulation; not the proposed litter mission.
[DroneVLA preprint](#)

Additional sources checked 26 September 2026. These references establish related work, not priority for the proposed contribution. Update the review before submission.